

Applied Machine Learning

Unit 06 — Lecture 16 Notes

Ensembles: Bagging, Boosting and Random Forests

Tofik Ali

Autumn 2026

Read this first

These notes are written so that you can learn the material *without* having been in the lecture. Every concept is developed from scratch, every number you see was produced by running the code shown beside it, and every figure was generated from real computation rather than drawn by hand.

The decision-tree lecture ended on a complaint. An unrestricted tree reaches training accuracy 1.0000 every single time and then falls over on new rows, and even a pruned tree is *unstable*: change a handful of training rows and the root split changes. That instability looked like a defect.

This lecture turns it into the raw material. If a learner produces a different model every time you perturb its training data, then you can build many of them, and their errors — being different errors — will partly cancel when you average. That is the whole of ensemble learning, and the rest is engineering: three different ways of forcing the members apart.

Two calculations carry the lecture and you should be able to do both with a pen. The first is Section 2: twenty-five independent voters, each 60% accurate, vote correctly with probability **0.846232** — and the same twenty-five voters at a mean pairwise correlation of 0.126 manage only 0.6932. The second is Section 7.3: three rounds of AdaBoost on ten points, with every weighted error, every α_t and every reweighting written out. Those two are the examinable arithmetic of this lecture.

Contents

1	How to use these notes	2
1.1	What you need	3
1.2	Learning outcomes	4
2	Why an ensemble beats its members	4
2.1	The setup, and the vocabulary	4
2.2	Condorcet's jury theorem, worked	5
2.3	The vote amplifies whatever the member already is	6
2.4	Independence is the whole game	6
2.5	Ensembles of classifiers: different learners, one vote	8
2.6	Three ways to manufacture disagreement	8
3	Bagging and the bootstrap	9
3.1	The bootstrap sample	9

3.2	The 63.2% result, derived	9
3.3	The bagging algorithm	10
3.4	Out-of-bag error: validation you did not have to pay for	11
4	Bagging reduces variance, not bias	12
4.1	What averaging can and cannot do	12
4.2	The experiment	12
4.3	The other half of the claim: bag a stump	13
5	Random forests	14
5.1	Bagging plus a random subset of features per split	14
5.2	Why decorrelation is the only lever left	14
5.3	Measured: worse trees, better forest	15
5.4	A forest against its own trees	16
6	Feature importance, honestly	17
6.1	Mean decrease in impurity	17
6.2	Demonstration 1: plant three useless columns	17
6.3	Demonstration 2: a table where it does mislead	18
6.4	Permutation importance	19
7	Boosting	20
7.1	Boosting inverts every design decision in bagging	20
7.2	The AdaBoost algorithm	20
7.3	Three rounds by hand	20
7.4	The combined classifier	22
7.5	The same thing from <code>AdaBoostClassifier</code>	22
7.6	Gradient boosting: fit the residuals	23
8	Bagging against boosting	25
8.1	The comparison	25
8.2	Demonstrated: flip the training labels	25
8.3	How many estimators?	26
8.4	All five, on three datasets	27
9	Summary	28
10	Practice questions	29
11	Solutions	30
12	References and further reading	34

1 How to use these notes

Read in order. Sections 2 to 5 are one continuous argument — why combining helps, how to make the members differ by resampling rows, what exactly that reduces, and how restricting the columns reduces it further — and nothing after Section 5 makes sense without them. Section 6 is a warning about the output people read most often and trust most wrongly. Sections 7 and 8 then develop the other family, boosting, and set the two families against each other.

Everything here assumes the decision-tree lecture: impurity, greedy top-down induction, overfitting, and the fact that information gain is biased towards attributes with many distinct values.

That last bias comes back in Section 6 wearing a different hat.

There are four kinds of box. **Key idea** — the one sentence to remember a month from now. **Worked example** — a calculation done by hand, with small numbers, before any library is used. **Caution** — the specific mistake that section exists to prevent. **Try it yourself** — something to run, with the expected output given so you can check.

Code appears in a framed block, and directly beneath it a shaded block showing what that code *actually printed*. If you run it and get something different, trust your machine and tell me.

1.1 What you need

```
pip install numpy scipy scikit-learn matplotlib
```

Every number in these notes was produced with:

```
import numpy, scipy, sklearn, matplotlib
print("numpy", numpy.__version__, "| scipy", scipy.__version__,
      "| sklearn", sklearn.__version__,
      "| matplotlib", matplotlib.__version__)
```

```
numpy 2.2.6 | scipy 1.15.3 | sklearn 1.7.2 | matplotlib 3.10.9
```

Four bundled datasets recur, and **none of them downloads anything**:

```
from sklearn.datasets import (load_breast_cancer, load_wine,
                              load_digits, load_iris)
from sklearn.model_selection import train_test_split
bc = load_breast_cancer(); Xb, yb = bc.data, bc.target
Xtr, Xte, ytr, yte = train_test_split(Xb, yb, test_size=0.3,
                                     random_state=0, stratify=yb)
```

```
load_breast_cancer 569 rows x 30 features, 2 classes
load_wine           178 rows x 13 features, 3 classes
load_digits         1797 rows x 64 features, 10 classes
load_iris           150 rows x 4 features, 3 classes
missing cells anywhere: 0
training split 398 rows, test split 171 rows
```

Unless a section says otherwise, `Xtr`, `ytr`, `Xte` and `yte` are exactly that split of `breast_cancer`: 398 training rows and 171 test rows. Two synthetic problems are also used — a one-dimensional regression target for Section 4 and a five-column designed table for Section 6 — and both are built in the code with a fixed seed.

One seed, and the two things that are not it

Every arbitrary random choice in these notes uses **one seed**, 0: the split above, the correlated-voter simulation, the bootstrap draws and the flipped labels are all `np.random.default_rng(0)` or `random_state=0`, and every listing that shuffles or samples shows that line rather than hiding it.

Two other kinds of number look like seeds and are not.

- A seed that *defines a dataset*. The three planted noise columns of Section 6 are built with `default_rng(42)`. Change it and you have different data, not a different experiment, so it stays where it is.
- A *sweep*. Section 3.4 fits over twenty splits and Section 8 over ten, with `random_state=s` running across the loop. The variation between runs *is* the demonstration there, so it is kept — but the sweep starts at 0, so it reproduces exactly too.

So every number printed in these notes reproduces exactly if you run the code as written on the library versions above. If you get something different, the difference is in your code or your versions, not in chance.

Do not reach for `fetch_*`

Every `fetch_*` loader downloads over HTTPS, and on the campus network those downloads fail with a certificate error you cannot fix from your machine. Nothing in this course depends on one: use the bundled loaders or a fixed-seed synthetic set.

1.2 Learning outcomes

These notes serve **CO4** — *apply and evaluate classification techniques, and select an appropriate model for a given problem*. By the end you should be able to:

1. State what an ensemble is, name its two requirements (members better than chance, members making different mistakes), and prove the first part of the claim with a binomial calculation.
CO4
2. Compute the majority-vote accuracy of m independent members of accuracy p by hand, and explain in one sentence why correlation between members destroys the gain. **CO4**
3. Describe bootstrap sampling, derive the $1 - 1/e = 0.632121$ result, and explain what the remaining 37% is used for. **CO4**
4. Use `oob_score_` as a validation estimate, and state its one caveat. **CO4**
5. Explain, with measured numbers, why bagging reduces variance and leaves bias where it was, and deduce what kind of base learner belongs inside a bag. **CO4**
6. Define a random forest as bagging plus a random feature subset per split, and use $\text{Var}(\bar{T}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ to explain what `max_features` buys. **CO4**
7. Interpret `feature_importances_` with its high-cardinality bias, and compute permutation importance instead. **CO4**
8. Execute AdaBoost by hand for two or three rounds: the weighted error, α_t , the reweighting and the combined sign. **CO4**
9. Describe gradient boosting as fitting the residuals of the current model. **CO4**
10. Choose between a forest and a boosted ensemble for a given problem and defend the choice in terms of label noise, tuning budget and parallelism. **CO4**

2 Why an ensemble beats its members

2.1 The setup, and the vocabulary

An **ensemble** is a set of models trained on the same problem, together with a rule for combining their predictions. The individual models are the **base learners** or **members**; the combination rule is the **aggregation**. For classification the usual aggregation is a **majority vote**; for regression it is the **mean**. Both have weighted versions, and Section 7 uses one.

Nothing in that definition says the members must be good. The surprising claim of this section is that they need only be *better than a coin*, and the argument is older than machine learning

by about two centuries: it is the Marquis de Condorcet's jury theorem of 1785, written about voters in an assembly rather than classifiers on a test set.

2.2 Condorcet's jury theorem, worked

Suppose m members each classify a given row correctly with probability p , and — this is the load-bearing assumption — suppose they do so **independently**. Let K be the number of members that get the row right. Independence makes K binomial:

$$\Pr[K = k] = \binom{m}{k} p^k (1 - p)^{m-k}, \quad (1)$$

and the majority vote is correct exactly when more than half the members are, so for odd m

$$\Pr[\text{vote correct}] = \sum_{k=\lceil m/2 \rceil}^m \Pr[K = k]. \quad (2)$$

Twenty-five voters, each 60% accurate

Take $m = 25$ and $p = 0.60$. A majority is at least 13, so we want $\sum_{k=13}^{25} \binom{25}{k} (0.6)^k (0.4)^{25-k}$. The first four terms:

k	$\binom{25}{k}$	$\binom{25}{k} (0.6)^k (0.4)^{25-k}$
13	5 200 300	0.11395006
14	4 457 400	0.14650722
15	3 268 760	0.16115794
16	2 042 975	0.15108557

and the last two are 0.00004738 at $k = 24$ and 0.00000284 at $k = 25$. Adding all thirteen terms gives

$$\Pr[\text{vote correct}] = \mathbf{0.846232}.$$

A single member scores 0.600000. The ensemble gains **+0.2462**, and not one member was improved — they were only combined.

Two sanity checks worth doing in your head. First, the largest term sits at $k = 15 = 0.6 \times 25$, which is the mean of the binomial; because the mean is above the majority threshold of 13, most of the probability mass is on the correct side, and that is the whole reason the answer is large. Second, if p were exactly 0.5 the distribution would be symmetric about 12.5 and the answer would be exactly 0.5 for every m .

```
from scipy.special import comb
from scipy import stats
n, p = 25, 0.60
terms = [(k, comb(n, k, exact=True) * p**k * (1 - p)**(n - k))
          for k in range(13, 26)]
maj = sum(t for _, t in terms)
print("majority (k >= 13) = %.6f" % maj)
print("scipy check = %.6f" % stats.binom.sf(12, n, p))
print("single member = %.6f" % p)
print("the ensemble gains %+.4f" % (maj - p))
```

k	C(25,k)	term
13	5200300	0.11395006
14	4457400	0.14650722
15	3268760	0.16115794
16	2042975	0.15108557
...		

```

24          25      0.00004738
25          1      0.00000284
majority (k >= 13) = 0.846232
scipy check      = 0.846232
single member    = 0.600000
the ensemble gains +0.2462

```

`stats.binom.sf(12, 25, 0.6)` is the survival function, $\Pr[K > 12]$, which is the same sum with a better name. It agrees to every digit printed.

2.3 The vote amplifies whatever the member already is

The jury theorem is a statement about a limit, and it cuts both ways.

```

print(" p 1 voter 11 voters 25 voters 101 voters")
for p in (0.40, 0.45, 0.49, 0.50, 0.51, 0.55, 0.60, 0.70):
    row = [stats.binom.sf(m // 2, m, p) for m in (1, 11, 25, 101)]
    print("%.2f %9.4f %9.4f %9.4f %9.4f" % (p, *row))

```

p	1 voter	11 voters	25 voters	101 voters
0.40	0.4000	0.2465	0.1538	0.0209
0.45	0.4500	0.3669	0.3063	0.1562
0.49	0.4900	0.4729	0.4598	0.4201
0.50	0.5000	0.5000	0.5000	0.5000
0.51	0.5100	0.5271	0.5402	0.5799
0.55	0.5500	0.6331	0.6937	0.8438
0.60	0.6000	0.7535	0.8462	0.9791
0.70	0.7000	0.9218	0.9825	1.0000

Read the table in three parts. Above $p = 0.5$ every row climbs towards 1; even $p = 0.51$ reaches 0.5799 with a hundred and one voters, it just takes a lot of them. At $p = 0.5$ nothing happens at all, for any m : the vote of a thousand coins is a coin. And below $p = 0.5$ the machinery runs in reverse — $p = 0.40$ falls to 0.0209, an ensemble that is confidently, reliably, systematically wrong.

Key idea

This is why the technical term is *weak* learner and not *bad* learner. A weak learner must be better than chance. How much better hardly matters; that it is on the right side of 0.5 matters completely.

2.4 Independence is the whole game

Equation (1) is only true because the members are independent. In practice they never are: they are trained on the same data, they see the same features, and when the data is genuinely hard they all fail on the same hard rows. So the honest question is not *does the jury theorem hold* but *how quickly does it decay as the members start to agree*.

To measure that we need voters whose accuracy is fixed at 0.60 but whose correlation we can dial. A Gaussian copula does it. Give voter i a latent score

$$Z_i = \sqrt{\rho} W + \sqrt{1 - \rho} E_i, \quad W, E_1, \dots, E_m \sim \mathcal{N}(0, 1) \text{ independent,}$$

and declare voter i correct when $Z_i < \Phi^{-1}(0.6)$. Each Z_i is standard normal whatever ρ is, so each voter is correct with probability exactly 0.60. The shared term W is a common cause — the difficulty of the row, if you like — and ρ controls how much of each voter's fate it decides.

```
rng = np.random.default_rng(0)
```

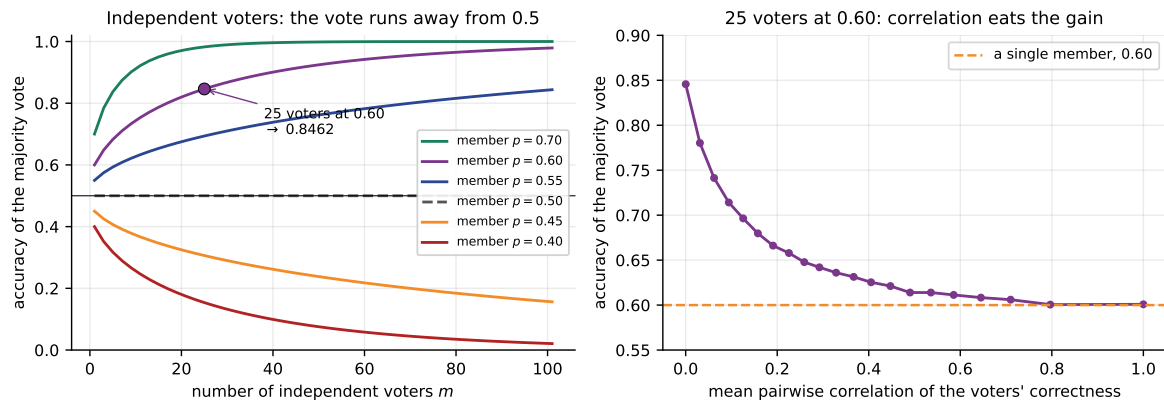


Figure 1: Left: the majority vote of m independent members, for six member accuracies. Above 0.5 the curves run to 1; below it they run to 0; at 0.5 nothing happens. Right: the same twenty-five members at $p = 0.60$, now correlated. The dashed line is a single member. **The left panel is the advertisement; the right panel is the contract.**

```

N, m, p = 200000, 25, 0.60
thr = stats.norm.ppf(p)
for rho in (0.0, 0.05, 0.1, 0.2, 0.4, 0.7, 1.0):
    W = rng.standard_normal((N, 1))
    E = rng.standard_normal((N, m))
    Z = np.sqrt(rho) * W + np.sqrt(1 - rho) * E
    correct = (Z < thr).astype(float) # 1 = this voter got it right
    C = np.corrcoef(correct, rowvar=False)
    rbar = (C.sum() - m) / (m * (m - 1))
    print(" %.2f %+.4f %.4f %.4f"
          % (rho, rbar, (correct.sum(1) > m / 2).mean(), correct.mean()))

```

rho_latent	mean pairwise corr	ensemble accuracy	members
0.00	+0.0001	0.8458	0.6002
0.05	+0.0310	0.7798	0.6006
0.10	+0.0627	0.7381	0.5993
0.20	+0.1260	0.6932	0.5995
0.40	+0.2595	0.6476	0.5997
0.70	+0.4883	0.6179	0.6002
1.00	+1.0000	0.5993	0.5993

Two hundred thousand simulated rows per line, so the third decimal is real. The first line reproduces the analytic 0.846232 to within simulation error, which is the check that the construction is right.

Then read the last column: it never moves. Every one of these ensembles is built from members that are exactly 60% accurate. And read the third column: a mean pairwise correlation of **0.126** — a number most people would describe as negligible — drops the vote from 0.8458 to **0.6932**, giving back 0.153 of the 0.246 the ensemble had won. By correlation 0.49 the committee is worth 0.6179, barely more than one member, and at correlation 1 it is worth exactly one member because it *is* one member, printed twenty-five times.

Key idea

An ensemble helps exactly to the extent that its members *fail on different rows*. Accuracy of the members is necessary; diversity of the members is what you are actually buying. Every technique in the rest of these notes is a way of manufacturing diversity.

2.5 Ensembles of classifiers: different learners, one vote

The most obvious way to get diverse members is to use genuinely different kinds of model. scikit-learn's `VotingClassifier` does this.

```
from sklearn.ensemble import VotingClassifier
members = [("knn", make_pipeline(StandardScaler(), KNeighborsClassifier(5))),
            ("tree", DecisionTreeClassifier(max_depth=3, random_state=0)),
            ("lr", make_pipeline(StandardScaler(),
                                LogisticRegression(max_iter=5000)))]
for name, mdl in members:
    mdl.fit(Xtr, ytr)
    print("%-5s test accuracy %.4f" % (name, accuracy_score(yte, mdl.predict(Xte))))
vote = VotingClassifier(members, voting="hard").fit(Xtr, ytr)
print("vote test accuracy %.4f" % accuracy_score(yte, vote.predict(Xte)))
```

```
knn   test accuracy 0.9474
tree  test accuracy 0.9006
lr    test accuracy 0.9591
vote  test accuracy 0.9474
mean pairwise disagreement between members 0.0702
rows where the members were not unanimous 18 of 171
```

Report the result you got, not the one you wanted

The vote scored 0.9474. Logistic regression alone scored 0.9591. **The ensemble did not beat its best member.** That is a real result and it is worth more to you than a tidy one. Three reasons, all visible in the output. Three members is far too few — go back to the table in Section 2 and notice how slowly $m = 11$ improves on $m = 1$. One member (the depth-3 tree, 0.9006) is much weaker than the others and drags a plain unweighted vote down. And most decisively, on 153 of the 171 test rows all three members already agree, so the vote has only 18 rows on which it can do anything at all; a single row is 0.0058 of the test accuracy here.

The lesson is not that voting does not work. It is that voting needs *many* members and *comparable* members. Bagging and boosting supply hundreds of both.

2.6 Three ways to manufacture disagreement

That leaves the design question: how do you produce hundreds of members that are all decent and all different? There are exactly three knobs, and the rest of this lecture is one section for each.

1. **Change the rows.** Give each member a different resample of the training set. That is **bagging**, Section 3.
2. **Change the columns.** Let each member — in fact each *split* of each member — see only a random subset of the features. Bagging plus this is a **random forest**, Section 5.
3. **Change the weights.** Make each member concentrate on the rows the committee so far gets wrong. That is **boosting**, Section 7.

What is *not* on the list: refitting the same deterministic learner on the same data with a different random seed. That gives $\rho = 1$ and, by the last row of the copula table, an ensemble worth exactly one member.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.; Bishop, *Pattern Recognition and Machine*

3 Bagging and the bootstrap

3.1 The bootstrap sample

A **bootstrap sample** of a training set of n rows is n rows drawn *with replacement* from it. Some rows are drawn twice or three times; some are not drawn at all. Both facts matter. Drawing *without* replacement would return the training set itself and every member would be identical; drawing with replacement produces a genuinely different training set each time, and it is the rows left out that will pay for the free validation in Section 3.4.

Five rows, four bags

Take the training set $\{A, B, C, D, E\}$ and draw five rows with replacement, four times:

bag	sample	distinct	out-of-bag
1	E D C B B	4/5	A
2	A A A A E	2/5	B, C, D
3	D E C D E	3/5	A, B
4	D D C C E	3/5	A, B

Two things to notice. The four bags are four *different* training sets: bag 2 is four copies of A and a single E , a training set that has never heard of three of the five rows. And every bag leaves somebody out — the model fitted on bag 3 has never seen A or B , so predicting A and B with that model is an honest out-of-sample prediction, obtained without holding anything back.

The distinct fractions here average $3/5 = 0.60$. The next subsection computes what that average should be.

3.2 The 63.2% result, derived

Fix one particular row. On a single draw the chance of *not* picking it is $1 - \frac{1}{n}$. The n draws are independent, so

$$\Pr[\text{row never drawn}] = \left(1 - \frac{1}{n}\right)^n, \quad \Pr[\text{row is in the bag}] = 1 - \left(1 - \frac{1}{n}\right)^n. \quad (3)$$

Since $\lim_{n \rightarrow \infty} (1 - 1/n)^n = e^{-1} = 0.367879$, the in-bag probability tends to

$$1 - e^{-1} = \mathbf{0.632121}.$$

By linearity of expectation the *expected fraction* of distinct rows in a bag is the same number, because each of the n rows is in the bag with that probability.

Check the formula at small n

At $n = 5$: $(1 - 1/5)^5 = 0.8^5 = 0.32768$, so the in-bag fraction is 0.67232. The four bags above averaged 0.60, a little below that, which is what four draws of a mean-0.672 quantity look like — with only four bags the sampling error is larger than the gap in either direction. At $n = 10$: $(0.9)^{10} = 0.348678$, so 0.651322. Notice how little the answer moved for a doubling of n : the convergence to 0.632121 is fast because $(1 - 1/n)^n = \exp(n \ln(1 - 1/n)) = \exp(-1 - \frac{1}{2n} - \frac{1}{3n^2} - \dots)$, so the error is $O(1/n)$ and already under 0.004 by $n = 100$.

```
rng = np.random.default_rng(0) # the one seed for this lecture
print(" n (1-1/n)^n 1-(1-1/n)^n empirical unique fraction")
```

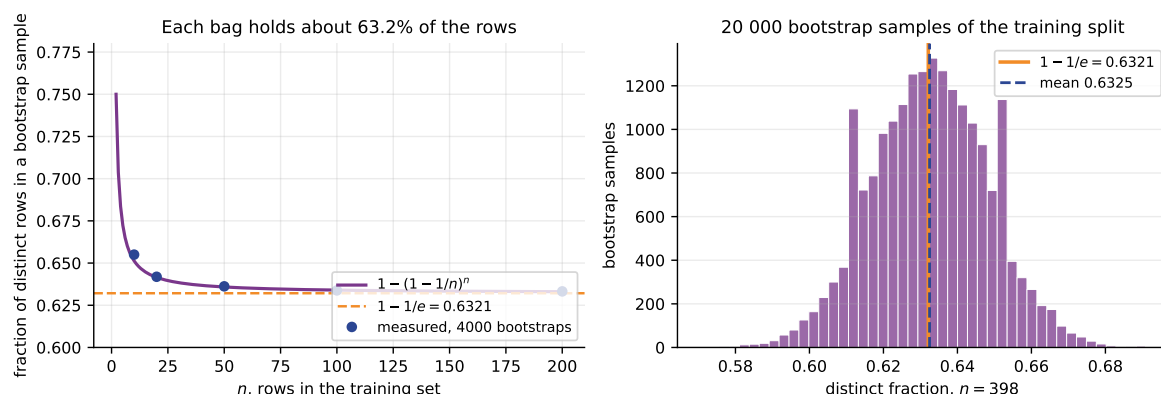


Figure 2: Left: the curve $1 - (1 - 1/n)^n$ against n , with the limit $1 - 1/e$ and five measured points from four thousand bootstraps each. Right: the distribution of the distinct fraction over twenty thousand bootstrap samples at $n = 398$. Mean 0.6325, standard deviation 0.0158.

```
for n in (5, 10, 50, 100, 1000, 10000):
    out = (1 - 1 / n) ** n
    emp = np.mean([len(np.unique(rng.integers(0, n, n))) / n
                    for _ in range(2000)])
    print("%6d %.6f %.6f %.6f" % (n, out, 1 - out, emp))
print("limit 1 - 1/e = %.6f" % (1 - np.exp(-1)))
```

n	$(1-1/n)^n$	$1-(1-1/n)^n$	empirical unique fraction
5	0.327680	0.672320	0.674900
10	0.348678	0.651322	0.655600
50	0.364170	0.635830	0.637460
100	0.366032	0.633968	0.634460
1000	0.367695	0.632305	0.632152
10000	0.367861	0.632139	0.632156

limit 1 - 1/e = 0.632121

Formula and measurement agree to four decimal places from $n = 50$ upwards. The right-hand panel of Figure 2 sharpens the point that 63.2% is a *mean*: over twenty thousand bootstrap samples of the 398-row training split the distinct fraction had mean 0.6325 and standard deviation 0.0158, with individual bags ranging from 0.5704 to 0.6910.

3.3 The bagging algorithm

Bagging

For $b = 1, \dots, B$: draw a bootstrap sample of the training set and fit a model T_b on it. Predict by majority vote (classification) or by the mean (regression) of $T_1, \dots, T_B$.

That is the whole algorithm. Breiman's 1996 paper that introduced it makes the condition explicit in its abstract: *the vital element is the instability of the prediction method*. Bagging helps a learner whose output changes a lot when the training data changes a little. Unpruned decision trees are the textbook case; ordinary least squares is close to the opposite, and bagging it buys almost nothing.

```
from sklearn.ensemble import BaggingClassifier
tree = DecisionTreeClassifier(random_state=0).fit(Xtr, ytr)
bag = BaggingClassifier(DecisionTreeClassifier(random_state=0),
                        n_estimators=200, oob_score=True,
                        random_state=0).fit(Xtr, ytr)
print("single unpruned tree test accuracy %.4f" % tree.score(Xte, yte))
print("bag of 200 such trees test accuracy %.4f" % bag.score(Xte, yte))
```

```
print("bag out-of-bag score %.4f" % bag.oob_score_)
```

```
single unpruned tree    test accuracy 0.9064
bag of 200 such trees   test accuracy 0.9298
bag out-of-bag score    0.9598
OOB minus test          +0.0300
5-fold CV on the training rows 0.9497
```

Two hundred copies of a learner that scores 0.9064 score 0.9298 together: +0.0234, or four test rows, for no change whatsoever to the learner. Same class, same hyperparameters, same features, same code. The only new thing is the resampling.

3.4 Out-of-bag error: validation you did not have to pay for

Section 3.2 showed that each member misses about 37% of the training rows. Turn that round: each training row is missed by about 37% of the members. Those members can be asked to predict it, and their prediction is a genuine out-of-sample prediction, because that row played no part in choosing any of their splits.

Out-of-bag score

For each training row, aggregate only the members for which that row was out-of-bag, and score the result against the truth. No rows are held back and no model is fitted twice.

There is one caveat and you should say it whenever you quote an OOB score: each OOB prediction comes from a sub-ensemble of roughly $0.37B$ members rather than all B , so for small B the OOB estimate is *pessimistic* — it measures a smaller ensemble than the one you will deploy. With B in the hundreds the effect is far below the noise.

Is it honest in practice? The single split above gave OOB 0.9598 against a test score of 0.9298 — optimistic by 0.0300, which looks alarming. One split proves nothing at all. Repeat over twenty splits.

```
d = []
for s in range(20):
    A, B, a, b = train_test_split(Xb, yb, test_size=0.3, random_state=s,
                                   stratify=yb)
    f = RandomForestClassifier(n_estimators=300, oob_score=True,
                              random_state=s).fit(A, a)
    d.append((f.oob_score_, f.score(B, b)))
d = np.array(d)
print("mean OOB score %.4f (sd %.4f)" % (d[:, 0].mean(), d[:, 0].std()))
print("mean held-out score %.4f (sd %.4f)" % (d[:, 1].mean(), d[:, 1].std()))
print("mean OOB - held-out %.4f" % (d[:, 0] - d[:, 1]).mean())
print("paired t = %.3f, p = %.3f" % stats.ttest_rel(d[:, 0], d[:, 1]))
```

```
20 random splits of breast_cancer, forest of 300 trees
mean OOB score      0.9584 (sd 0.0054)
mean held-out score 0.9576 (sd 0.0137)
mean OOB - held-out +0.0008
paired t = 0.209, p = 0.836
```

The systematic bias is +0.0008, about a seventh of a single test row, and the paired t -test cannot distinguish it from zero ($p = 0.836$). Note also the standard deviations: 0.0054 for OOB against 0.0137 for the held-out score. The free estimate is not merely unbiased here, it is *less noisy*, because it is computed on all 398 training rows while the held-out score is computed on only 171.

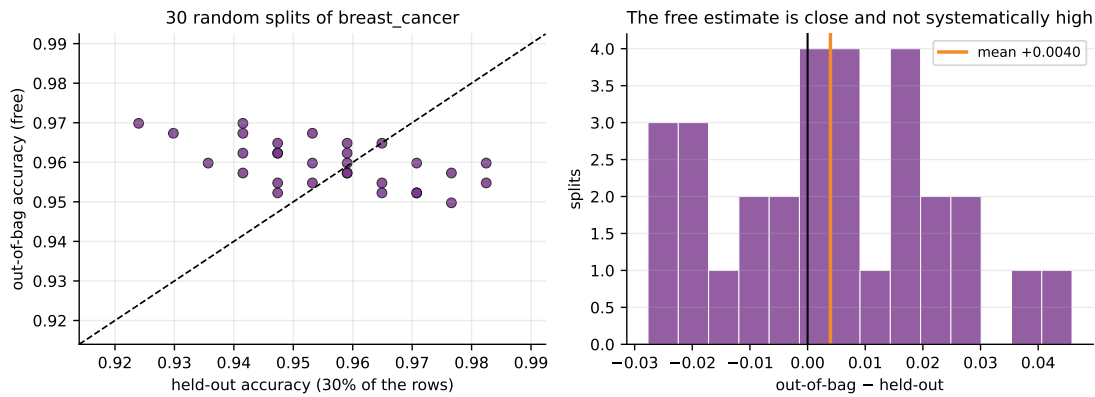


Figure 3: Thirty random splits of `breast_cancer`, forests of 300 trees. Left: OOB accuracy against held-out accuracy, with the line $y = x$. Right: the distribution of their difference, centred on +0.0040 over these thirty splits.

What OOB is and is not

OOB estimates the *generalisation accuracy of this ensemble*. It is a legitimate substitute for a validation set — you may tune `max_features` on it, as Section 5 does. It is *not* a substitute for a final held-out test set if you have tuned on it, for exactly the reason cross-validation is not: once you select a hyperparameter by maximising a score, that score is optimistic. And OOB is only available when `bootstrap=True`; set `bootstrap=False` and there is no out-of-bag anything.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.; Zhang et al., *Dive into Deep Learning*, Ch. 5.

4 Bagging reduces variance, not bias

4.1 What averaging can and cannot do

For squared-error regression, the expected error of a fitted model $\hat{f}$ at a point x decomposes exactly:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{bias}^2} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{noise}}, \quad (4)$$

where the expectations are over training sets drawn from the same distribution.

Averaging B models leaves the expectation alone — the mean of B copies of a random quantity has the same mean as one copy — and shrinks the variance. So bagging can move the second term of (4) and nothing else. That is a prediction, and it is testable.

4.2 The experiment

Take $f(x) = \sin(4x) + x$ on $[0, 3]$, draw 120 training points uniformly with Gaussian noise of standard deviation $\sigma = 0.35$, and repeat the whole thing 300 times. On each of the 300 training sets fit (a) one unpruned regression tree and (b) a bag of 50 of them. Then, at each of 60 evaluation points, take the mean and the variance of the 300 predictions. Because we know f exactly, both terms of (4) can be computed rather than estimated.

```
def truth(x):
    return np.sin(4 * x) + x

xg = np.linspace(0.05, 2.95, 60); fg = truth(xg)
```

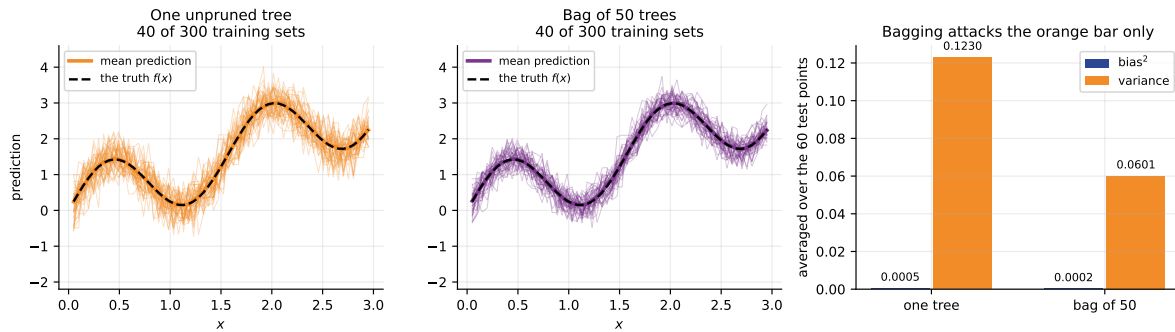


Figure 4: Left and centre: forty of the three hundred fitted curves, with the mean prediction in bold and the true $f(x)$ dashed. The single trees scatter; the bagged curves hug the truth. Crucially the two bold lines are in the *same place* — the bias did not move. Right: the two terms of (4) as bars.

REP, NTR, SD = 300, 120, 0.35

```
def sweep(make):
    rng = np.random.default_rng(0)
    P = np.zeros((REP, len(xg)))
    for r in range(REP):
        xt = rng.uniform(0, 3, NTR)
        yt = truth(xt) + rng.normal(0, SD, NTR)
        P[r] = make(r).fit(xt[:, None], yt).predict(xg[:, None])
    return P

pt = sweep(lambda r: DecisionTreeRegressor(random_state=r))
pb = sweep(lambda r: BaggingRegressor(DecisionTreeRegressor(random_state=r),
                                     n_estimators=50, random_state=r))

for name, P in (("single deep tree", pt), ("bag of 50 trees", pb)):
    bias2 = np.mean((P.mean(0) - fg) ** 2)
    var = np.mean(P.var(0))
    print("%-16s bias^2 %.4f variance %.4f bias^2+var %.4f"
          % (name, bias2, var, bias2 + var))
```

```
single deep tree bias^2 0.0005 variance 0.1230 bias^2+var 0.1235
bag of 50 trees bias^2 0.0002 variance 0.0601 bias^2+var 0.0603
variance fell to 48.8% of the single tree's; bias^2 moved -0.0003
irreducible noise variance 0.1225
```

Variance $0.1230 \rightarrow 0.0601$: halved. Bias² $0.0005 \rightarrow 0.0002$: both numbers are negligible beside the variance, which is the point — a deep tree is nearly unbiased to begin with, so there was nothing there for bagging to fix even if it could. The noise floor $\sigma^2 = 0.35^2 = 0.1225$ is printed as a reminder that no method can drive the total below it.

4.3 The other half of the claim: bag a stump

Halving a variance that was already the dominant term is a good look for bagging. To see that it really cannot touch bias, run the identical experiment with a learner that is *badly* biased: a depth-1 stump, which can produce only two output values and therefore cannot represent a sine wave at all.

```
ps = sweep(lambda r: DecisionTreeRegressor(max_depth=1, random_state=r))
pbs = sweep(lambda r: BaggingRegressor(
    DecisionTreeRegressor(max_depth=1, random_state=r),
    n_estimators=50, random_state=r))
```

```
single stump bias^2 0.1856 variance 0.0281 bias^2+var 0.2137
bag of 50 stumps bias^2 0.1842 variance 0.0147 bias^2+var 0.1989
```

bagging removed 48% of the stump's variance and left the bias where it was

The variance halves again — $0.0281 \rightarrow 0.0147$, the same 48%, which is reassuring because the mechanism is identical. And the bias moves by **0.0014**, less than one percent of itself. Fifty stumps are still a stump.

Key idea

Bagging is a variance reducer and nothing else. Two consequences follow immediately. **(1)** Put *deep, unpruned* learners inside a bag — you want the high-variance, low-bias end, because variance is what bagging removes. Pruning the members throws away the wrong half. **(2)** If your model is *underfitting*, bagging it will not help. Fixing bias is boosting's job, which is why boosting exists and why its members are shallow.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.; Zhang et al., *Dive into Deep Learning*, Ch. 5.

5 Random forests

5.1 Bagging plus a random subset of features per split

A **random forest** is bagging with one extra restriction:

Random forest

Grow B trees on B bootstrap samples. At *every split of every tree*, choose `max_features` of the d columns at random and search only those for the best split. Vote.

The default `max_features` is $\lfloor \sqrt{d} \rfloor$ for classification and $d/3$ for regression. For `breast_cancer`, $\sqrt{30} = 5.4772$, so scikit-learn's `'sqrt'` uses 5 features per split.

The motivation is Section 2.4. If one feature is much more predictive than the others, every bagged tree will put it at the root, and probably the same feature second as well. The trees then look alike, their errors correlate, and by the right-hand panel of Figure 1 an ensemble of near-identical members is worth barely more than one member. Forbidding the dominant feature at a random $(d - \text{max_features})/d$ of the splits forces those trees to find other structure.

5.2 Why decorrelation is the only lever left

Suppose the B members each have variance σ^2 and every pair has correlation ρ . Then

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B T_b\right) = \frac{1}{B^2} \left(B\sigma^2 + B(B-1)\rho\sigma^2 \right) = \rho\sigma^2 + \frac{1-\rho}{B} \sigma^2. \quad (5)$$

Read equation (5) carefully

The second term has B in the denominator and goes to zero as you add members. **The first term does not contain B at all.**

At $B = 100$, as a fraction of one member's variance:

ρ	$\rho + (1 - \rho)/B$
0.00	0.0100
0.10	0.1090
0.30	0.3070
0.50	0.5050
0.80	0.8020
1.00	1.0000

At $\rho = 0.5$, a hundred trees and a million trees are both stuck at approximately half the variance of one tree. Once B is large the *only* remaining way to reduce the variance is to reduce ρ , and `max_features` is the knob that does it.

5.3 Measured: worse trees, better forest

The prediction is that lowering `max_features` lowers both the correlation between trees *and* the accuracy of each tree, so the ensemble should be best somewhere in the middle. Measure the correlation as the mean pairwise Pearson correlation of the members' predicted probabilities on the test rows.

```
def tree_corr(forest, X):
    """Mean pairwise Pearson correlation of the member trees' P(class 1)."""
    P = np.array([t.predict_proba(X)[:, 1] for t in forest.estimators_])
    C = np.corrcoef(P); k = len(P)
    return (C.sum() - k) / (k * (k - 1))

for label, mf in (("bagging (max_features=30)", None),
                  ("forest (max_features=5)", "sqrt"),
                  ("forest (max_features=1)", 1)):
    f = RandomForestClassifier(n_estimators=100, max_features=mf,
                              random_state=0).fit(Xtr, ytr)
    single = np.mean([t.score(Xte, yte) for t in f.estimators_])
    print("%-27s tree corr %.4f mean single tree %.4f ensemble %.4f"
          % (label, tree_corr(f, Xte), single, f.score(Xte, yte)))
```

bagging (max_features=30)	tree corr 0.8261	mean single tree 0.9130	ensemble 0.9240
forest (max_features=5)	tree corr 0.7945	mean single tree 0.9110	ensemble 0.9532
forest (max_features=1)	tree corr 0.7010	mean single tree 0.8861	ensemble 0.9415

Exactly as predicted, and it is worth reading the three columns separately. Correlation falls monotonically, $0.8261 \rightarrow 0.7945 \rightarrow 0.7010$. Mean member accuracy falls monotonically too, $0.9130 \rightarrow 0.9110 \rightarrow 0.8861$. And the ensemble — the product of the two competing effects — peaks in the middle at **0.9532**. At `max_features=1` the trees have been decorrelated the most but have become too weak to carry the vote.

Sweeping the whole range confirms the shape and, more usefully, shows that you can find the optimum without ever touching the test set.

```
for mf in (1, 2, 3, 5, 8, 12, 20, 30):
    f = RandomForestClassifier(n_estimators=200, max_features=mf,
                              oob_score=True, random_state=0).fit(Xtr, ytr)
    single = np.mean([t.score(Xte, yte) for t in f.estimators_])
    print(" %2d %.4f %.4f %.4f %.4f"
          % (mf, tree_corr(f, Xte), single, f.oob_score_, f.score(Xte, yte)))
```

max_features	tree corr	mean single tree	OOB	test
1	0.7046	0.8864	0.9673	0.9357
2	0.7582	0.9030	0.9673	0.9532

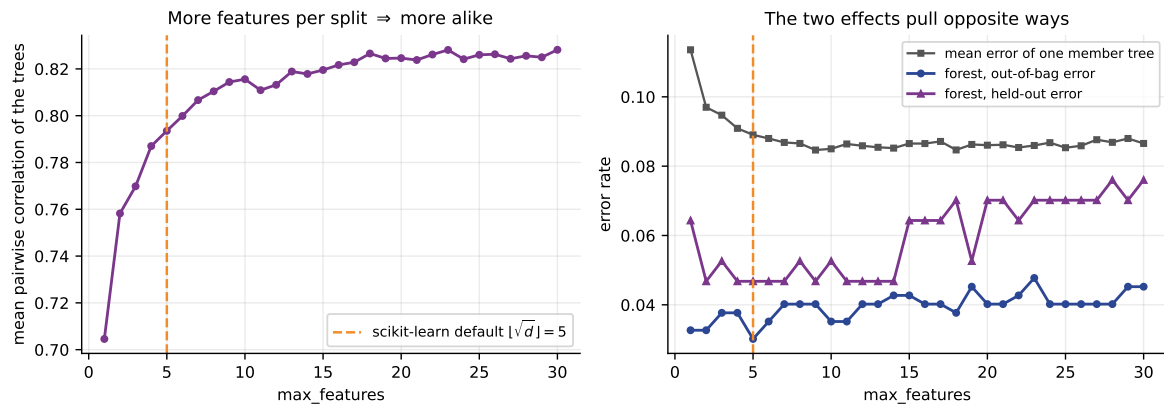


Figure 5: Left: mean pairwise tree correlation rises steadily with `max_features`. Right: the error of one member falls with `max_features` while the ensemble error is minimised in the middle. Dashed line: the default $\lfloor \sqrt{30} \rfloor = 5$.

3	0.7698	0.9053	0.9623	0.9474
5	0.7935	0.9110	0.9698	0.9532
8	0.8104	0.9135	0.9598	0.9474
12	0.8132	0.9141	0.9598	0.9532
20	0.8246	0.9139	0.9598	0.9298
30	0.8282	0.9135	0.9548	0.9240

The OOB column is minimised at `max_features=5` — OOB error 0.0302, against 0.0452 at 30. That is the scikit-learn default, and it was selected here using only training rows. The test column is noisier (it is 171 rows, so one row is 0.0058) but tells the same story: everything from 2 to 12 is good, and the plain-bagging end at 30 is clearly worse.

5.4 A forest against its own trees

One last measurement, because it is the whole lecture in five lines.

```
f = RandomForestClassifier(n_estimators=300, oob_score=True,
                           random_state=0).fit(Xtr, ytr)
print("mean over all 300 trees %.4f"
      % np.mean([t.score(Xte, yte) for t in f.estimators_]))
print("worst single tree %.4f"
      % min([t.score(Xte, yte) for t in f.estimators_]))
print("the forest %.4f" % f.score(Xte, yte))
```

```
a forest is 300 trees; the first three have 13, 16, 18 leaves
their individual test accuracies 0.8889 0.9123 0.8772
mean over all 300 trees          0.9120
worst single tree                0.8480
the forest                      0.9532
```

Not one of the three hundred members is a model you would ship. Each is a small overfitted tree, grown on 63% of the rows, allowed only five features per split; the average scores 0.9120 and the worst scores 0.8480. The forest scores 0.9532, above the average member by 0.0412 and above every one of the first three shown. That is Figure 1's left panel happening on real data.

Watch the plateau

Fit forests with `n_estimators` in $\{1, 5, 25, 100, 400\}$ on the split above and print the test accuracy each time. You should see it rise steeply and then stop: on the noisy split of Section 8 the numbers were 0.6959, 0.8012, 0.9123, 0.9298, 0.9298. **More trees in a**

forest never makes it worse in expectation — equation (5) says the variance of the average is monotone decreasing in B — it only makes it slower. On a 171-row test set expect a jitter of a row or two either way on top of that. So the honest answer to “how many trees” is “as many as you can afford, and then stop worrying”.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.; Zhang et al., *Dive into Deep Learning*, Ch. 5.

6 Feature importance, honestly

6.1 Mean decrease in impurity

`RandomForestClassifier.feature_importances_` reports the **mean decrease in impurity** (MDI). For a single tree, each split reduces the impurity of the node it acts on; weight that reduction by the fraction of training rows reaching the node, add up over all splits that used a given feature, and you have that feature’s importance in that tree. Average over the trees and normalise so the vector sums to 1.

It is popular because it is free — the numbers are a by-product of fitting, requiring no extra data and no extra computation. And it is biased.

Impurity importance favours high-cardinality features

A feature with many distinct values offers many candidate split points. More candidates means a better *best* candidate, purely by chance, even when the feature carries no information at all. So MDI systematically scores high-cardinality columns above low-cardinality ones.

This is the same bias the decision-tree lecture found in information gain, where a 14-value `Day` column scored the maximum possible gain and built a useless tree. The forest inherits it. scikit-learn’s own user guide states it in as many words: impurity-based importance is “strongly biased” and “favor[s] high cardinality features”.

6.2 Demonstration 1: plant three useless columns

Add to `breast_cancer` three columns drawn from a random number generator: a coin (2 values), a die (6 values) and a row identifier (569 distinct values, a permutation of $0 \dots 568$). None of them has any relationship to the diagnosis.

```
rng = np.random.default_rng(42); nb = len(Xb)
coin = rng.integers(0, 2, nb).astype(float) # 2 levels, pure noise
dice = rng.integers(1, 7, nb).astype(float) # 6 levels, pure noise
uid = rng.permutation(nb).astype(float) # 569 levels, pure noise
Xn = np.c_[Xb, coin, dice, uid]
A, B, a, b = train_test_split(Xn, yb, test_size=0.3, random_state=0, stratify=yb)
f = RandomForestClassifier(n_estimators=500, random_state=0).fit(A, a)
imp = f.feature_importances_
```

```
forest test accuracy with the three noise columns added 0.9357
top 5 by impurity importance:
  worst perimeter      0.1404
  worst concave points 0.1200
  mean concave points  0.1161
  worst radius         0.1067
  worst area           0.0912
the three planted noise columns:
```

```

noise_uid(569)          0.0028   rank 30 of 33
noise_dice(6)           0.0012   rank 32 of 33
noise_coin(2)           0.0005   rank 33 of 33
the useless 569-level column scores 5.9 times the useless 2-level column
it outranks 1 of the 30 real clinical measurements
top 3 by permutation importance (held-out rows):
worst radius            +0.0029
worst concave points    +0.0023
worst area              +0.0010
noise_uid(569)          +0.0000   (sd 0.0000)
noise_dice(6)           -0.0006   (sd 0.0018)
noise_coin(2)           +0.0000   (sd 0.0000)

```

Be precise about what this shows and what it does not. The bias is real and measurable: the three noise columns rank in exactly cardinality order, and the 569-level column scores 5.9 times the 2-level column despite both being the same kind of nothing. But the thirty clinical measurements are genuinely strong, so the noise still lands at ranks 30, 32 and 33 out of 33, and outranks only one real measurement. **On this dataset the bias distorts the numbers; it does not mislead you about which columns matter.**

Report that honestly. Then build a case where it does mislead.

6.3 Demonstration 2: a table where it does mislead

Eight hundred rows and five columns. `signal_binary` takes two values and agrees with y 75% of the time; that is the only relationship in the data, so no model can exceed 0.7500 accuracy. The other four columns — a coin, a die, a uniform draw and a permutation — are pure noise, and the last two have 800 distinct values each.

```

rng = np.random.default_rng(0); n = 800
sig = rng.integers(0, 2, n)
ys = np.where(rng.random(n) < 0.25, 1 - sig, sig) # 75% agreement
Xs = np.c_[sig, # 2 levels, informative
           rng.integers(0, 2, n), # 2 levels, useless
           rng.integers(1, 7, n), # 6 levels, useless
           rng.random(n), # 800 levels, useless
           rng.permutation(n)].astype(float) # 800 levels, useless
A, B, a, b = train_test_split(Xs, ys, test_size=0.3, random_state=0, stratify=ys)
f = RandomForestClassifier(n_estimators=500, random_state=0).fit(A, a)
pi = permutation_importance(f, B, b, n_repeats=30, random_state=0)

```

```

y agrees with signal_binary 75% of the time; nothing else matters
forest test accuracy 0.7333 (the best any model can do is 0.7500)
           impurity  permutation
signal_binary(2)    0.2715    +0.2136 (sd 0.0294)
noise_coin(2)       0.0237    +0.0083 (sd 0.0115)
noise_dice(6)       0.0829    +0.0189 (sd 0.0131)
noise_cont(800)     0.3110    +0.0081 (sd 0.0145)
noise_uid(800)      0.3109    +0.0114 (sd 0.0127)
impurity importance puts TWO useless columns above the only informative one
useless 800-level column / useless 2-level column = 13.1 times

```

Read the impurity column top to bottom. A uniform random draw scores 0.3110 and a row identifier scores 0.3109; the only column carrying any information scores 0.2715 and comes *third*. A student who ranked the features by impurity importance and kept the top two would keep both noise columns and throw away the signal. Among the four useless columns the ordering is again pure cardinality: 0.0237, 0.0829, 0.3110, 0.3109 for 2, 6, 800, 800 levels — a factor of 13.1 between the two-level and eight-hundred-level versions of exactly the same nothing.

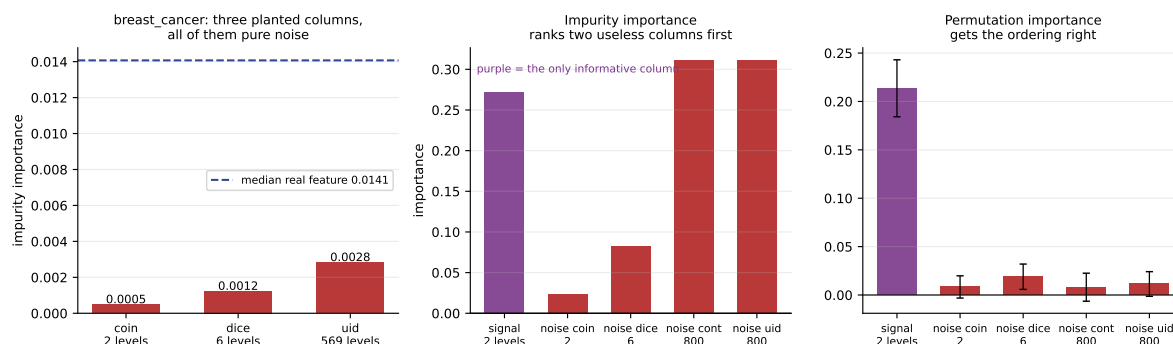


Figure 6: Left: on `breast_cancer`, the impurity importance of the three planted columns rises with cardinality although all three are pure noise; the dashed line is the median real feature. Centre and right: on the designed table, impurity puts two noise columns above the informative one and permutation importance does not. Red is noise in every panel.

6.4 Permutation importance

Permutation importance

Score the fitted model on **held-out** rows. Randomly shuffle one column, destroying its relationship with y while leaving its marginal distribution alone, and score again. The drop is that column's importance. Repeat several times and average.

It cannot be fooled by cardinality because it never looks at the tree structure. It asks only the question you actually care about: *if this column were destroyed, would the predictions get worse?* Shuffling a row identifier changes nothing the model depends on, so the score does not fall and the importance is about zero.

The right-hand columns of the output above are exactly that measurement, with thirty repeats. `signal_binary` scores +0.2136 against +0.0081 and +0.0114 for the two 800-level noise columns — a separation of about 19 $\times$, in the correct direction, with standard deviations small enough (0.0294 against 0.0145) that the ordering is not an accident.

Two things permutation importance still cannot do

Compute it on held-out rows. On training rows an overfitted model depends heavily on columns that do not generalise, and you will measure the overfitting rather than the signal. **Beware correlated features.** If two columns carry the same information, shuffling either one leaves the model able to read the other, so both score near zero and both look useless. On `breast_cancer`, whose thirty features are strongly intercorrelated (`worst radius`, `worst perimeter` and `worst area` are nearly the same measurement), this is exactly why the permutation importances in Demonstration 1 are so small — +0.0029 for the best feature. That is a real limitation, not a bug, and the usual remedy is to cluster correlated features and permute a cluster at a time.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.; Zhang et al., *Dive into Deep Learning*, Ch. 5.

7 Boosting

7.1 Boosting inverts every design decision in bagging

Bagging trains its members in parallel, on resampled rows, deep, and gives them an equal vote. Boosting trains them in sequence, on reweighted rows, shallow, and gives them a weighted vote. The reason for every one of those inversions is the same: boosting attacks **bias**, which is the half of equation (4) that bagging cannot reach.

Boosting

Fit a weak learner. Increase the weight of the rows it got wrong. Fit another weak learner on the reweighted data. Repeat. Predict with a weighted vote in which better members count for more.

Because each new member is trained specifically on what the committee still gets wrong, a boosted ensemble of stumps — models far too simple to fit anything on their own — can drive training error to zero. Section 7.3 shows that happening on ten points.

7.2 The AdaBoost algorithm

Labels are $y_i \in \{-1, +1\}$. Initialise $w_i = 1/n$ for all n training rows. For $t = 1, \dots, T$:

1. Fit a weak learner h_t to the training data weighted by w .

2. Compute the weighted error $\varepsilon_t = \sum_{i: h_t(x_i) \neq y_i} w_i$.

3. Compute the member weight

$$\alpha_t = \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t}. \quad (6)$$

4. Reweight and renormalise:

$$w_i \leftarrow \frac{w_i e^{-\alpha_t y_i h_t(x_i)}}{Z_t}, \quad Z_t = \sum_j w_j e^{-\alpha_t y_j h_t(x_j)}. \quad (7)$$

Predict with $\text{sign } F(x)$ where $F(x) = \sum_{t=1}^T \alpha_t h_t(x)$.

Two things to read carefully. In (6), α_t is large when ε_t is small: a member that solved a hard weighted problem is trusted more. It is exactly zero at $\varepsilon_t = 0.5$ (a member no better than chance gets no vote) and negative below that, which is the algorithm correctly deciding to use a worse-than-chance member backwards.

In (7), the product $y_i h_t(x_i)$ is $+1$ when the member is right and -1 when it is wrong, so the multiplier is $e^{-\alpha_t} < 1$ for correct rows and $e^{+\alpha_t} > 1$ for incorrect ones. One expression does both jobs, and Z_t puts the weights back on the simplex.

7.3 Three rounds by hand

Ten points on a line: $x = 0, 1, \dots, 9$ with labels

$$y = +1, +1, +1, -1, -1, -1, +1, +1, +1, -1.$$

The weak learner is a decision stump: pick a threshold θ and a polarity $s \in \{+1, -1\}$, and predict s when $x < \theta$ and $-s$ otherwise. No single stump can classify these ten points, because the labels change sign three times and a stump changes sign once.

Round 1

All weights are $w_i = 0.1$. Search every threshold and both polarities and count weighted errors. The best is $\theta = 2.5$, $s = +1$: predict +1 for $x \in \{0, 1, 2\}$ and -1 for $x \in \{3, \dots, 9\}$. That is right on 0, 1, 2 (label +1), right on 3, 4, 5, 9 (label -1), and wrong on 6, 7, 8 (label +1, predicted -1). Three wrong rows, each of weight 0.1:

$$\varepsilon_1 = 0.1 + 0.1 + 0.1 = \mathbf{0.3000}.$$

Then

$$\alpha_1 = \frac{1}{2} \ln \frac{1 - 0.3}{0.3} = \frac{1}{2} \ln 2.33333 = \frac{1}{2} (0.847298) = \mathbf{0.423649}.$$

Reweight. Wrong rows are multiplied by $e^{+0.423649} = 1.527525$, giving $0.1 \times 1.527525 = 0.152753$ each. Right rows are multiplied by $e^{-0.423649} = 0.654654$, giving 0.065465 each. The normaliser is

$$Z_1 = 3(0.152753) + 7(0.065465) = 0.458258 + 0.458258 = 0.916515.$$

(That the two halves come out equal is not a coincidence: AdaBoost's α is chosen precisely so that after reweighting the misclassified rows carry total weight exactly 0.5.) Dividing,

$$w_{6,7,8} = \frac{0.152753}{0.916515} = \mathbf{0.1667}, \quad w_{\text{others}} = \frac{0.065465}{0.916515} = \mathbf{0.0714}.$$

Check: $3(0.1667) + 7(0.0714) = 0.5001 + 0.4998 = 0.9999 \approx 1$. Good.

Rounds 2 and 3

Round 2 starts with 6, 7, 8 heavy at 0.1667 and everything else at 0.0714. The best stump is now $\theta = 8.5$, $s = +1$: predict +1 for $x < 8.5$ and -1 for $x = 9$. It gets 0, 1, 2 right, 6, 7, 8 right (which is what the heavy weights were pushing for), 9 right, and 3, 4, 5 wrong. Those three now carry 0.0714 each:

$$\varepsilon_2 = 3(0.0714) = \mathbf{0.2143}, \quad \alpha_2 = \frac{1}{2} \ln \frac{0.7857}{0.2143} = \mathbf{0.649642}.$$

Reweight makes 3, 4, 5 the heavy rows at 0.1667, drops 6, 7, 8 to 0.1061 and 0, 1, 2, 9 to 0.0455.

Round 3 therefore arrives with 3, 4, 5 heavy, and the best stump *flips polarity*: $\theta = 5.5$, $s = -1$, predicting -1 for $x < 5.5$ and $+1$ for $x \geq 5.5$. It gets 3, 4, 5 right and 6, 7, 8 right, and is wrong on 0, 1, 2 (weight 0.0455 each) and 9 (weight 0.0455):

$$\varepsilon_3 = 4(0.0455) = \mathbf{0.1818}, \quad \alpha_3 = \frac{1}{2} \ln \frac{0.8182}{0.1818} = \mathbf{0.752039}.$$

The pattern to take away: ε falls, $0.3000 \rightarrow 0.2143 \rightarrow 0.1818$, and α rises, $0.4236 \rightarrow 0.6496 \rightarrow 0.7520$. Each round faces a harder weighted problem than the last and is trusted more for solving it.

```
def best_stump(x, y, w):
    best = None
    for t in np.arange(-0.5, 10, 1.0):
        for s in (+1, -1):
            h = np.where(x < t, s, -s).astype(float)
            err = w[h != y].sum()
            if best is None or err < best[0] - 1e-12:
                best = (err, t, s, h)
```

```

return best

x = np.arange(10).astype(float)
y = np.array([1, 1, 1, -1, -1, -1, 1, 1, 1, -1]).astype(float)
w = np.full(10, 0.1)
alphas, hs = [], []
for rnd in range(1, 4):
    err, t, s, h = best_stump(x, y, w)
    alpha = 0.5 * np.log((1 - err) / err)
    alphas.append(alpha); hs.append(h)
    w = w * np.exp(-alpha * y * h); w = w / w.sum()

round 1: stump h(x) = +1 if x < 2.5 else -1
weighted error eps = 0.3000
alpha = 0.5*ln((1-eps)/eps) = 0.4236
misclassified points x = 6, 7, 8
new weights 0.0714 0.0714 0.0714 0.0714 0.0714 0.1667 0.1667 0.1667 0.0714
sign(F) so far gets 7 of 10 right
round 2: stump h(x) = +1 if x < 8.5 else -1
weighted error eps = 0.2143
alpha = 0.5*ln((1-eps)/eps) = 0.6496
misclassified points x = 3, 4, 5
new weights 0.0455 0.0455 0.0455 0.1667 0.1667 0.1667 0.1061 0.1061 0.1061 0.0455
sign(F) so far gets 7 of 10 right
round 3: stump h(x) = -1 if x < 5.5 else +1
weighted error eps = 0.1818
alpha = 0.5*ln((1-eps)/eps) = 0.7520
misclassified points x = 0, 1, 2, 9
new weights 0.1250 0.1250 0.1250 0.1019 0.1019 0.1019 0.0648 0.0648 0.0648 0.1250
sign(F) so far gets 10 of 10 right

```

7.4 The combined classifier

Now assemble $F(x) = 0.4236 h_1(x) + 0.6496 h_2(x) + 0.7520 h_3(x)$.

x	f1	f2	f3	F(x)	sign	y
0	+1.0000	+1.0000	-1.0000	+0.3213	+1	+1
1	+1.0000	+1.0000	-1.0000	+0.3213	+1	+1
2	+1.0000	+1.0000	-1.0000	+0.3213	+1	+1
3	-1.0000	+1.0000	-1.0000	-0.5260	-1	-1
4	-1.0000	+1.0000	-1.0000	-0.5260	-1	-1
5	-1.0000	+1.0000	-1.0000	-0.5260	-1	-1
6	-1.0000	+1.0000	+1.0000	+0.9780	+1	+1
7	-1.0000	+1.0000	+1.0000	+0.9780	+1	+1
8	-1.0000	+1.0000	+1.0000	+0.9780	+1	+1
9	-1.0000	-1.0000	+1.0000	-0.3213	-1	-1

training errors of the 3-round ensemble: 0
training errors of the best single stump: 3

Check two rows with a pen. At $x = 0$: $+0.4236 + 0.6496 - 0.7520 = +0.3212$, positive, so class +1, which is right. At $x = 6$: $-0.4236 + 0.6496 + 0.7520 = +0.9780$, positive, class +1, right. Look down the f_1, f_2, f_3 columns: not one of them matches the y column, and each of the three individually gets at least three points wrong. Their weighted sum gets all ten.

7.5 The same thing from AdaBoostClassifier

```

from sklearn.ensemble import AdaBoostClassifier
ada = AdaBoostClassifier(DecisionTreeClassifier(max_depth=1, random_state=0),
                        n_estimators=3, random_state=0).fit(x[:, None], y)
print("sklearn estimator_errors_ ", ada.estimator_errors_)

```

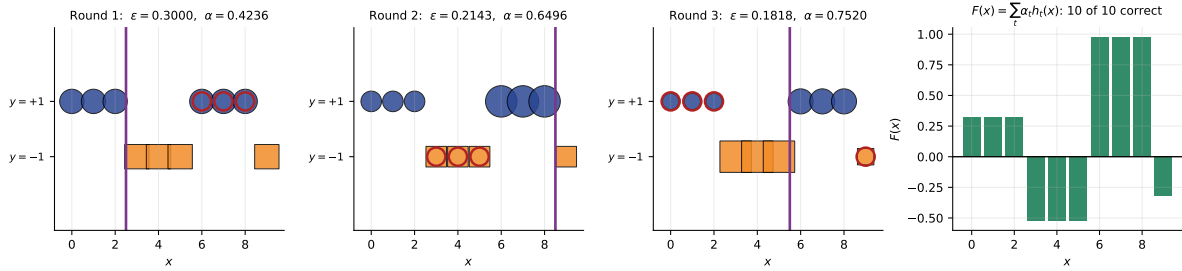


Figure 7: Three rounds of AdaBoost. Marker area is the row’s current weight; the vertical line is the stump’s threshold; red circles are that round’s mistakes — and they are exactly the markers that grow in the next panel. Right: $F(x)$, green where its sign matches y .

```
print("sklearn estimator_weights_ ", ada.estimator_weights_)
```

```
sklearn estimator_errors_ 0.3000 0.2143 0.1818
hand-computed eps        0.3000 0.2143 0.1818
sklearn estimator_weights_ 0.8473 1.2993 1.5041
hand-computed alpha       0.4236 0.6496 0.7520
ratio sklearn/hand        2.0000 2.0000 2.0000
sklearn training errors 0, stump thresholds [2.5, 8.5, 5.5]
```

Every weighted error matches to four decimals and all three thresholds match exactly — 2.5, 8.5, 5.5, the same stumps in the same order. The member weights differ by **exactly a factor of two, every time**, and that is not a discrepancy to be explained away.

scikit-learn implements SAMME, the multiclass generalisation of AdaBoost, whose member weight is

$$\alpha_t^{\text{SAMME}} = \ln \frac{1 - \varepsilon_t}{\varepsilon_t} + \ln(K - 1)$$

for K classes. With $K = 2$ the second term is $\ln 1 = 0$ and the first is $2\alpha_t$. Multiplying every member weight by the same positive constant cannot change $\text{sign} \sum_t \alpha_t h_t(x)$, so the two ensembles make identical predictions — both get all ten right.

7.6 Gradient boosting: fit the residuals

AdaBoost reweights rows because for classification that is what “pay more attention to this row” means. For squared-error regression there is a more direct move: fit the next member to **what is left over**.

Gradient boosting, squared error

Start with $F_0(x) = \bar{y}$. For $m = 1, 2, \dots$: compute the residuals $r_i = y_i - F_{m-1}(x_i)$, fit a small tree g_m to (x_i, r_i) , and set $F_m = F_{m-1} + \eta g_m$.

The name comes from the observation that for squared-error loss $L = \frac{1}{2} \sum_i (y_i - F(x_i))^2$, the negative gradient with respect to the prediction at row i is $-\partial L / \partial F(x_i) = y_i - F(x_i)$, which *is* the residual. Boosting is therefore gradient descent carried out in the space of functions, one small tree per step, and other loss functions give other pseudo-residuals.

Three stages on eight points

Take $x = 1, \dots, 8$ and $y = 5, 5, 6, 6, 9, 9, 10, 10$, with $\eta = 1$ and depth-1 stumps.
 $F_0 = \bar{y} = 7.5$. The residuals are $-2.5, -2.5, -1.5, -1.5, +1.5, +1.5, +2.5, +2.5$ and the initial SSE = $4(2.5^2) + 4(1.5^2)$ — no, count them properly: $2(2.5^2) + 2(1.5^2) + 2(1.5^2) + 2(2.5^2) = 12.5 + 4.5 + 4.5 + 12.5 = 34$.

Stage 1. The best depth-1 split of those residuals is at $x < 4.5$; the left leaf holds $-2.5, -2.5, -1.5, -1.5$ with mean -2.0 and the right leaf holds $+1.5, +1.5, +2.5, +2.5$ with mean $+2.0$. So $F_1 = 5.5, 5.5, 5.5, 5.5, 9.5, 9.5, 9.5, 9.5$ and $\text{SSE} = 4(0.5^2) + 4(0.5^2) = 2.0$.

Stage 2. The new residuals are $-0.5, -0.5, +0.5, +0.5, -0.5, -0.5, +0.5, +0.5$. The best split is $x < 2.5$: left mean -0.5 , right mean $(0.5 + 0.5 - 0.5 - 0.5 + 0.5 + 0.5)/6 = 0.1\bar{6}$. So $F_2 = 5.0, 5.0, 5.667, 5.667, 9.667, 9.667, 9.667, 9.667$ and $\text{SSE} = 1.3333$.

Stage 3. Split at $x < 6.5$ gives $F_3 = 4.889, 4.889, 5.556, 5.556, 9.556, 9.556, 10.000, 10.000$ and $\text{SSE} = 1.0370$.

```
xr = np.arange(1, 9).astype(float)
yr = np.array([5.0, 5.0, 6.0, 6.0, 9.0, 9.0, 10.0, 10.0])
Fm = np.full(8, yr.mean())
for m in range(1, 4):
    r = yr - Fm
    st = DecisionTreeRegressor(max_depth=1, random_state=0).fit(xr[:, None], r)
    Fm = Fm + st.predict(xr[:, None])
    print(" SSE %.4f" % np.sum((yr - Fm) ** 2))
```

```
stage 0: F0 = mean(y) = 7.5000
  residuals -2.500 -2.500 -1.500 -1.500 +1.500 +1.500 +2.500 +2.500
stage 1: stump on the residuals, split at x < 4.5
  stump output -2.000 -2.000 -2.000 -2.000 +2.000 +2.000 +2.000 +2.000
  F1           +5.500 +5.500 +5.500 +5.500 +9.500 +9.500 +9.500 +9.500
  SSE 2.0000
stage 2: stump on the residuals, split at x < 2.5
  stump output -0.500 -0.500 +0.167 +0.167 +0.167 +0.167 +0.167 +0.167
  F2           +5.000 +5.000 +5.667 +5.667 +9.667 +9.667 +9.667 +9.667
  SSE 1.3333
stage 3: stump on the residuals, split at x < 6.5
  stump output -0.111 -0.111 -0.111 -0.111 -0.111 -0.111 +0.333 +0.333
  F3           +4.889 +4.889 +5.556 +5.556 +9.556 +9.556 +10.000 +10.000
  SSE 1.0370
```

```
from sklearn.ensemble import GradientBoostingRegressor
gbm = GradientBoostingRegressor(n_estimators=3, learning_rate=1.0,
                                max_depth=1, random_state=0).fit(xr[:, None], yr)
print("max abs difference %.2e" % np.max(np.abs(gbm.predict(xr[:, None]) - Fm)))
```

```
sklearn init_.constant_ 7.5000 (hand: mean = 7.5000)
sklearn predictions +4.889 +4.889 +5.556 +5.556 +9.556 +9.556 +10.000 +10.000
hand F3             +4.889 +4.889 +5.556 +5.556 +9.556 +9.556 +10.000 +10.000
max abs difference  0.00e+00
```

Not approximately equal — *bit-for-bit* identical. The library’s initial constant is the mean, and its three stages are the three stumps you just fitted by hand.

The learning rate

η was set to 1.0 above so that the hand computation and the library would agree exactly.

Do not use $\eta = 1$ in practice. The default is 0.1 and 0.05 is common: add a small multiple of each stump, and use many more of them. Small steps generalise better than a few large ones, and η and $\mathbf{n_estimators}$ trade off against each other — halving η roughly doubles the number of stages you need. Unlike a forest, where more trees is always safe, $\mathbf{n_estimators}$ is a real hyperparameter here, and Section 8 shows what happens if you get it wrong.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.; Bishop, *Pattern Recognition and Machine*

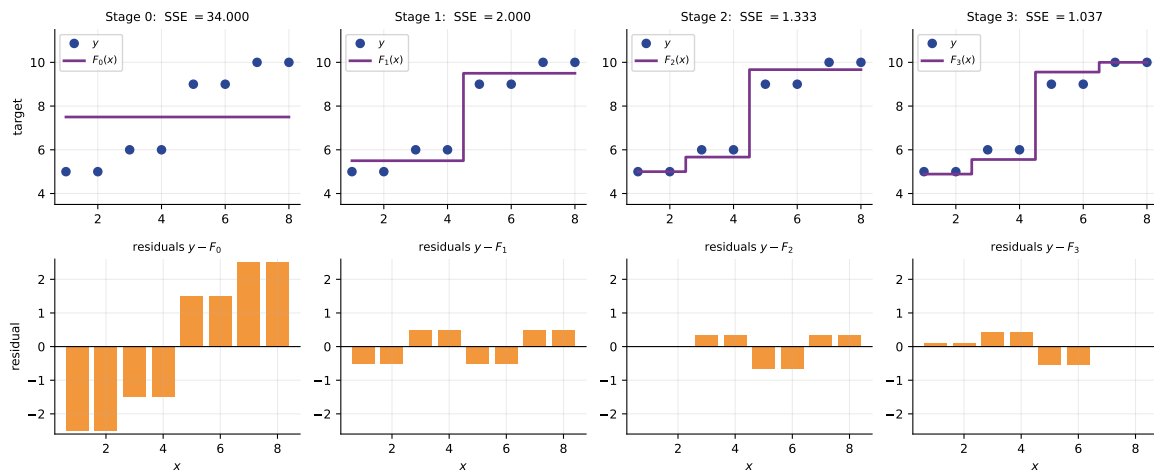


Figure 8: Gradient boosting on eight points. Top: the staircase F_m closing on the data, with the sum of squared errors in each title. Bottom: the residuals $y - F_m$ that the next stump will be fitted to.

Learning, Ch. 4–5, 7, 14.;Zhang et al., *Dive into Deep Learning*, Ch. 5.

8 Bagging against boosting

8.1 The comparison

	Bagging / random forests	Boosting
Members trained	in parallel, independently	in sequence, on the last's errors
Data each member sees	a bootstrap resample	the same rows, reweighted
Member depth	deep, unpruned	shallow: stumps or depth 2–3
Aggregation	equal vote	weighted by α_t
Attacks	variance	bias
More members	plateaus; safe	can overfit; must be watched
Label noise	robust: averages it away	fragile : chases it
Parallelisable	yes, trivially	no, inherently sequential
Tuning needed	almost none	η , depth, T
Free validation	yes (oob_score_)	no

The row that matters most in practice is label noise. A mislabelled training row is a row that every reasonable member gets wrong — which is precisely the signal AdaBoost uses to decide a row deserves more weight. Boosting will keep increasing that row's weight until the ensemble is partly built around it. Bagging has no such mechanism; the mislabelled row is one vote among n in each bag and is averaged away.

8.2 Demonstrated: flip the training labels

Flip a fraction of the *training* labels at random, leave the test labels clean, and average over ten splits.

```
for frac in (0.00, 0.10, 0.20, 0.30, 0.40):
    accs = []
    for s in range(10): # splits 0..9: the sweep is derived
        A, B, a, b = train_test_split(Xb, yb, test_size=0.3, random_state=s,
                                       stratify=yb) # from the lecture seed 0
        rr = np.random.default_rng(1000 + s) # flip stream, clear of the split
        a2 = a.copy()
        k = int(round(frac * len(a2)))
        if k:
            fl = rr.choice(len(a2), k, replace=False)
```

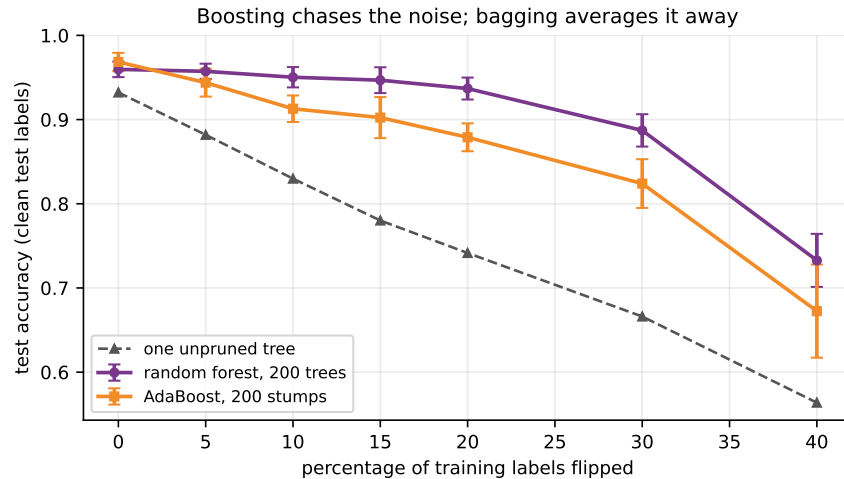


Figure 9: Test accuracy on clean labels against the percentage of training labels flipped, averaged over ten splits with error bars at one standard deviation. AdaBoost starts higher and crosses below the forest almost immediately.

```

a2[f1] = 1 - a2[f1]
rf = RandomForestClassifier(n_estimators=200, random_state=s).fit(A, a2)
ab = AdaBoostClassifier(DecisionTreeClassifier(max_depth=1,
                                             random_state=s),
                       n_estimators=200, random_state=s).fit(A, a2)
dt = DecisionTreeClassifier(random_state=s).fit(A, a2)
accs.append((rf.score(B, b), ab.score(B, b), dt.score(B, b)))
print(" %3d%% %.4f %.4f %.4f"
      % (int(frac * 100), *np.mean(accs, axis=0)))

```

noise	RandomForest(200)	AdaBoost(200 stumps)	single tree
0%	0.9596	0.9684	0.9322
10%	0.9503	0.9129	0.8298
20%	0.9368	0.8789	0.7415
30%	0.8871	0.8240	0.6661
40%	0.7327	0.6725	0.5637

Say the first line before you say anything else: **at 0% noise AdaBoost is better**, by 0.0088. On clean data, boosting is usually the stronger method, and a lecture that only showed the noisy columns would be teaching you a false generalisation.

Then follow the columns down. By 10% noise the forest is ahead by 0.0374, by 20% by 0.0579, and by 40% by 0.0602. The crossover happens almost immediately, somewhere below 10% — which is well inside the range of label error you should expect in real hand-annotated data.

Note also the third column. The single unpruned tree collapses fastest of all, from 0.9322 to 0.5637, barely above chance. Both ensembles beat it everywhere by a wide margin. The argument of this section is forest versus boosting; the argument of the whole lecture is ensemble versus not.

8.3 How many estimators?

The second practical difference is what happens as T grows. Take one split, flip 25% of its training labels, and track both methods stage by stage. Both the split and the flips come from the lecture's one seed:

```

A, B, a, b = train_test_split(Xb, yb, test_size=0.3, random_state=0,
                              stratify=yb)
rr = np.random.default_rng(0) # the one seed for this lecture

```

```
a2 = a.copy()
fl = rr.choice(len(a2), int(0.25 * len(a2)), replace=False)
a2[fl] = 1 - a2[fl] # flip a quarter of the labels
```

```
25% of the training labels flipped
  trees      forest test acc  AdaBoost test acc
    1         0.6959         0.9006
    5         0.8012         0.9006
   10         0.8129         0.9006
   25         0.9123         0.9064
   50         0.9298         0.8480
  100         0.9298         0.8480
  200         0.9298         0.8129
  400         0.9298         0.8070
  800         0.9240         0.7719
forest:  best 0.9298 at T=50, final 0.9240, drop from peak -0.0058
AdaBoost: best 0.9064 at T=19, final 0.7719, drop from peak -0.1345
AdaBoost training accuracy at T=800: 0.9648
```

The forest climbs and flattens. Between $T = 50$ and $T = 800$ it moves by 0.0058, which on 171 test rows is exactly one row — noise. AdaBoost peaks at $T = 19$ with 0.9064 and then loses **0.1345** over the remaining seven hundred and eighty-one rounds, ending far below where it started at $T = 1$.

The last line explains why. At $T = 800$ AdaBoost's *training* accuracy is 0.9648 on labels that are 25% wrong. It has memorised the flipped labels, which is what the reweighting rule instructs it to do.

On clean labels the same experiment is much kinder to boosting:

```
clean labels, same split
AdaBoost: best 0.9766 at T=324, final 0.9591, drop from peak -0.0175
forest:    best 0.9532 at T=100, final 0.9415, drop from peak -0.0117
```

AdaBoost reaches 0.9766, above the forest's best 0.9532, and gives back 0.0175 — three test rows — by $T = 800$. The forest gives back 0.0117, two test rows, from its own peak. Neither of those is a trend: on a 171-row test set one row is 0.0058, and equation (5) makes the *variance* of the averaged prediction monotone decreasing in B , so the forest's curve can only jitter, not decline. The asymmetry is in the shape of the two curves rather than in a single number. The forest is flat to within a couple of rows from $T = 50$ onwards on both panels; AdaBoost is flat on the clean panel but loses 0.1345 on the noisy one and is still falling at $T = 800$. With a forest, T is a budget; with boosting it is a hyperparameter that has to be chosen, ideally by watching a validation curve and stopping early.

8.4 All five, on three datasets

```
cvk = StratifiedKFold(5, shuffle=True, random_state=0)
models = [("single tree", DecisionTreeClassifier(random_state=0)),
          ("bagging 200", BaggingClassifier(DecisionTreeClassifier(random_state=0),
                                                                    n_estimators=200, random_state=0)),
          ("forest 200", RandomForestClassifier(n_estimators=200, random_state=0)),
          ("AdaBoost 200", AdaBoostClassifier(
              DecisionTreeClassifier(max_depth=1, random_state=0),
              n_estimators=200, random_state=0)),
          ("grad boosting", GradientBoostingClassifier(random_state=0))]
```

	wine	breast	digits
single tree	0.9273	0.9262	0.8592
bagging 200	0.9608	0.9579	0.9533
forest 200	0.9832	0.9667	0.9766

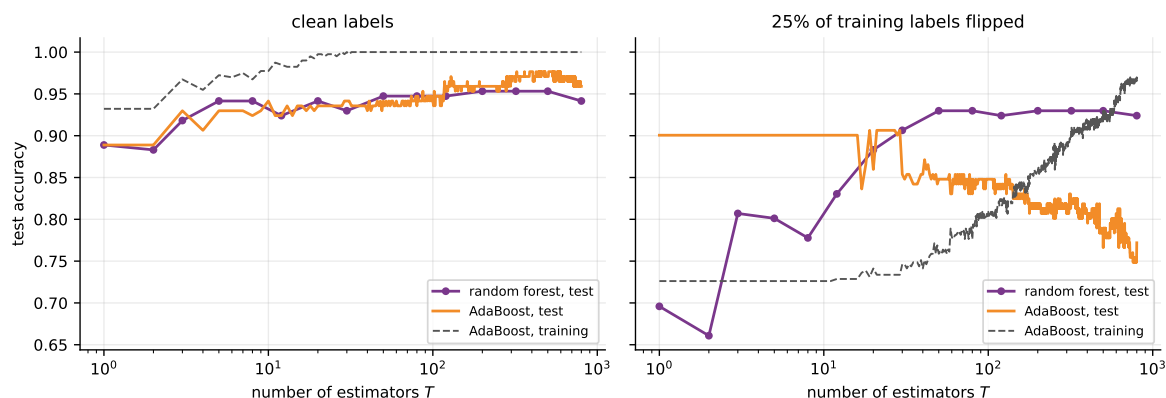


Figure 10: Test accuracy against the number of estimators, log scale. Left: clean labels. Right: 25% of training labels flipped. The dashed grey line is AdaBoost’s *training* accuracy, which climbs in both panels.

AdaBoost 200	0.9665	0.9754	0.8458
grad boosting	0.9494	0.9649	0.9655

Every ensemble beats the single tree on every dataset, with one exception, and the exception is the interesting entry: AdaBoost scores 0.8458 on **digits**, *below* the single tree’s 0.8592.

Boosted stumps are a two-class instrument

digits has ten classes. SAMME requires each member to beat random guessing on the weighted problem, that is $\varepsilon_t < 1 - 1/K = 0.9$, and a depth-1 stump on a ten-class problem is barely above that line. The member weights α_t come out tiny and the ensemble never gets going. Give AdaBoost a depth-3 or depth-5 base learner on a multiclass problem, or use gradient boosting, which scores 0.9655 on the same data. The forest, which needed no thought at all, scores 0.9766.

Key idea

On tabular data, fit a random forest with default settings first. It needs no feature scaling, no tuning and no separate validation split (use `oob_score=True`), it is top or near-top on all three datasets above, and it tells you within seconds what accuracy is achievable. Then — if the labels are trustworthy and you have a tuning budget — try gradient boosting to beat it.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.; Zhang et al., *Dive into Deep Learning*, Ch. 5.

9 Summary

1. An **ensemble** combines many models. It helps only when the members are better than chance and *fail on different rows*.
2. **Condorcet’s jury theorem**. Twenty-five independent voters at $p = 0.60$ vote correctly with probability **0.846232**, a gain of +0.2462. Below $p = 0.5$ the same machinery makes things worse: 0.40 becomes 0.0209 with 101 voters. At $p = 0.5$ nothing happens.
3. **Correlation is the enemy**. With members held at exactly 60% accuracy, a mean pairwise correlation of 0.126 drops the vote from 0.8458 to **0.6932**; at correlation 1 the committee is worth one member, 0.5993.

4. **The bootstrap.** A sample of n drawn with replacement from n contains $1 - (1 - 1/n)^n \rightarrow 1 - 1/e = \mathbf{0.632121}$ of the distinct rows. Measured at $n = 10\,000$: 0.632156. At $n = 398$ the mean over 20 000 bags was 0.6325 with standard deviation 0.0158.
5. **Out-of-bag.** The remaining 37% gives free validation. Over twenty splits, OOB minus held-out was $+\mathbf{0.0008}$ with $p = 0.836$, and OOB had the *smaller* standard deviation (0.0054 against 0.0137).
6. **Bagging reduces variance only.** Over 300 resamples the variance of a deep tree fell $0.1230 \rightarrow \mathbf{0.0601}$ while bias^2 moved $0.0005 \rightarrow 0.0002$. Bagging fifty *stumps* halved the variance ($0.0281 \rightarrow 0.0147$) and left bias^2 at $0.1856 \rightarrow 0.1842$. So put deep learners in a bag, and do not bag an underfitting model.
7. **The decorrelation identity.** $\text{Var}(\bar{T}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. The first term contains no B , so once B is large the only lever left is ρ .
8. **Random forests pull ρ down with `max_features`:** $0.8261 \rightarrow 0.7945 \rightarrow 0.7010$ at 30, 5, 1 features. Members get *worse* ($0.9130 \rightarrow 0.8861$) and the ensemble peaks in the middle at $\mathbf{0.9532}$. Mean member 0.9120, worst member 0.8480, forest 0.9532. OOB picked `max_features=5` without seeing the test set.
9. **Impurity importance is biased** towards high-cardinality columns. On a designed table two pure-noise 800-level columns scored 0.3110 and 0.3109 against 0.2715 for the only informative one. Permutation importance on held-out rows scored $+\mathbf{0.2136}$ against $+0.0081$ and $+0.0114$.
10. **AdaBoost.** $\varepsilon_t = \sum_{h_t(x_i) \neq y_i} w_i$, $\alpha_t = \frac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t}$, $w_i \leftarrow w_i e^{-\alpha_t y_i h_t(x_i)} / Z_t$. On ten points: $\varepsilon = 0.3000, 0.2143, 0.1818$ and $\alpha = 0.4236, 0.6496, 0.7520$; three stumps that each get three points wrong combine to get **10** of 10 right. scikit-learn reproduces every ε and threshold; its weights are exactly 2α because SAMME omits the $\frac{1}{2}$.
11. **Gradient boosting** fits the residuals: $F_0 = 7.5$, then stumps on $y - F_{m-1}$, SSE $34 \rightarrow 2 \rightarrow 1.333 \rightarrow \mathbf{1.037}$, matching `GradientBoostingRegressor` to 0.00e+00. Use $\eta \approx 0.1$, not 1.
12. **Bagging versus boosting.** Parallel against sequential, variance against bias, robust against fragile. At 0% label noise AdaBoost wins by 0.0088; at 40% the forest wins by $\mathbf{0.0602}$. With 25% flipped labels AdaBoost peaks at $T = 19$ and then loses 0.1345 while its training accuracy climbs to 0.9648; the forest's peak-to-final drop is 0.0058, one test row.

10 Practice questions

Work these with a pen before running anything. Solutions with full working follow in Section 11.

1. **Five independent voters.** Five classifiers, each correct with probability $p = 0.70$, independent. Compute the accuracy of their majority vote exactly. Then say what happens to your answer if the five are in fact perfectly correlated.
2. **Bootstrap, by hand.** A training set has $n = 4$ rows. (a) Compute exactly the probability that a given row is *absent* from a bootstrap sample, as a fraction and as a decimal. (b) Compute the expected number of distinct rows in the bag. (c) Compare with the large- n limit and say whether $n = 4$ is above or below it, and why.
3. **AdaBoost, round 1 by hand.** Five points, $x = 1, 2, 3, 4, 5$, with labels $y = +1, +1, -1, +1, -1$. The weak learner is a stump of the form “predict s if $x < \theta$, else $-s$ ”, with θ a half-integer.

Starting from uniform weights $w_i = 0.2$: find the stump with the lowest weighted error, and compute ε_1 , α_1 and the five normalised weights after the update. Give α_1 to four decimals.

4. **AdaBoost, round 2 by hand.** Continue Question 3. Using the weights you computed, find the best stump for round 2 and compute ε_2 and α_2 . Then evaluate $F(x) = \alpha_1 h_1(x) + \alpha_2 h_2(x)$ at all five points and state how many the two-round ensemble classifies correctly.
5. **The decorrelation identity.** A forest has $B = 200$ trees, each of variance $\sigma^2 = 1$, with mean pairwise correlation $\rho = 0.40$. (a) What is the variance of the forest's average? (b) How much would you gain by going to $B = 2000$? (c) How much would you gain by halving ρ to 0.20 at $B = 200$? Which is the better investment?
6. **Reading an OOB score.** You fit a forest of 300 trees and get `oob_score_` = 0.9598 and a held-out test score of 0.9298 on 171 rows. A classmate says the OOB score is dishonest and should be discarded. Using the numbers in Section 3.4, give two reasons why that conclusion does not follow, and say what you would do to settle it.
7. **Reading an importance table.** A colleague fits a forest to a customer dataset and sends you the top four features by `feature_importances_`: `transaction_id` (0.31), `timestamp` (0.24), `amount` (0.18), `is_weekend` (0.02). What do you tell them, what single computation do you ask for, and what do you expect it to show?
8. **Which ensemble, and why.** You have 50 000 rows of tabular medical data labelled by three junior annotators whose pairwise agreement is about 85%. You have one afternoon and a sixteen-core machine. Forest or gradient boosting? Justify with at least two specific numbers from these notes.
9. **Gradient boosting by hand.** Four points, $x = 1, 2, 3, 4$ with $y = 2, 4, 8, 10$. Take $F_0 = \bar{y}$ and $\eta = 1$, and fit one depth-1 stump to the residuals (splitting between consecutive x values, leaf value = mean of the residuals in the leaf). Compute F_1 and the sum of squared errors before and after.
10. **Why bagging fails here.** Explain, in terms of equation (4), why bagging an ordinary least-squares linear regression on a large dataset buys almost nothing, and name one situation in which bagging a linear model *would* help.

11 Solutions

1. **Five independent voters.** A majority of five is at least three. With $p = 0.7$, $q = 0.3$:

$$\binom{5}{3} p^3 q^2 = 10(0.343)(0.09) = 0.30870,$$

$$\binom{5}{4} p^4 q = 5(0.2401)(0.3) = 0.36015, \quad \binom{5}{5} p^5 = 0.16807.$$

Sum: $0.30870 + 0.36015 + 0.16807 = \mathbf{0.83692}$. The vote gains +0.13692 over a single member.

If the five are perfectly correlated they are the same classifier five times over: whenever one is right all are right, so the majority is right exactly when a member is, and the accuracy is **0.70**. The entire gain disappears, and no member's accuracy changed — this is the copula table of Section 2.4 in miniature.

2. Bootstrap by hand at $n = 4$. (a) Each of the four draws misses a fixed row with probability $3/4$, and the draws are independent, so

$$\Pr[\text{absent}] = \left(\frac{3}{4}\right)^4 = \frac{81}{256} = \mathbf{0.316406}.$$

Therefore $\Pr[\text{present}] = 175/256 = 0.683594$.

(b) Let I_j indicate that row j is present. Expectation is linear whether or not the I_j are independent (they are not), so

$$\mathbb{E}[\#\text{distinct}] = \sum_{j=1}^4 \Pr[I_j = 1] = 4 \times \frac{175}{256} = \frac{175}{64} = \mathbf{2.734375},$$

a fraction of 0.683594 of the four rows.

(c) The limit is $1 - 1/e = 0.632121$, so $n = 4$ is *above* it. The sequence $1 - (1 - 1/n)^n$ decreases towards the limit from above: at $n = 2$ it is 0.75, at $n = 4$ it is 0.6836, at $n = 10$ it is 0.6513. Small bags retain a larger share of their rows because with few rows a given row gets a relatively larger share of the draws.

3. AdaBoost round 1. Points $x = 1, \dots, 5$, labels $y = +1, +1, -1, +1, -1$, weights all 0.2.

Enumerate the stumps. With $s = +1$ (predict +1 below θ):

θ	predictions on $x = 1..5$	wrong rows
0.5	−, −, −, −, −	1, 2, 4 (3)
1.5	+, −, −, −, −	2, 4 (2)
2.5	+, +, −, −, −	4 (1)
3.5	+, +, +, −, −	3, 4 (2)
4.5	+, +, +, +, −	3 (1)
5.5	+, +, +, +, +	3, 5 (2)

The $s = -1$ stumps are the complements and get $5 - k$ wrong, so their best is $5 - 3 = 2$; they cannot beat one error. Two stumps tie at a single error: $\theta = 2.5$ and $\theta = 4.5$. Take $\theta = 2.5$, $s = +1$ (the first found by a left-to-right search). It is wrong only on $x = 4$, so

$$\varepsilon_1 = 0.2 = \mathbf{0.2000}, \quad \alpha_1 = \frac{1}{2} \ln \frac{0.8}{0.2} = \frac{1}{2} \ln 4 = \frac{1}{2}(1.386294) = \mathbf{0.693147}.$$

(That $\alpha_1 = \ln 2$ exactly is a happy accident of $\varepsilon = 0.2$.)

Reweight. $e^{0.693147} = 2$ and $e^{-0.693147} = 0.5$, so the wrong row becomes $0.2 \times 2 = 0.4$ and each of the four right rows becomes $0.2 \times 0.5 = 0.1$. The normaliser is $Z_1 = 0.4 + 4(0.1) = 0.8$. Hence

$$w_4 = \frac{0.4}{0.8} = \mathbf{0.5}, \quad w_1 = w_2 = w_3 = w_5 = \frac{0.1}{0.8} = \mathbf{0.125}.$$

Check: $0.5 + 4(0.125) = 1$. And note the misclassified row now carries total weight exactly 0.5, as the algorithm always arranges.

4. AdaBoost round 2. Weights are now 0.125, 0.125, 0.125, 0.5, 0.125. Recount the weighted errors of the same six thresholds, both polarities. The single point $x = 4$ carries half the mass, so any stump that gets it wrong has $\varepsilon \geq 0.5$ and is useless. Stumps that get $x = 4$ right:

- $\theta = 3.5$, $s = -1$: predicts −, −, −, +, +; wrong on 1, 2, 5; $\varepsilon = 0.125 \times 3 = 0.375$.

- $\theta = 4.5$, $s = +1$: predicts $+, +, +, +, -$; wrong on 3; $\varepsilon = 0.125$.
- $\theta = 5.5$, $s = +1$: predicts $+, +, +, +, +$; wrong on 3, 5; $\varepsilon = 0.25$.
- $\theta = 2.5$, $s = +1$ (round 1's stump): wrong on 4; $\varepsilon = 0.5$ — exactly chance, and $\alpha = 0$. The algorithm has made the previous member worthless, which is the point of the reweighting.

The best is $\theta = 4.5$, $s = +1$, with

$$\varepsilon_2 = \mathbf{0.1250}, \quad \alpha_2 = \frac{1}{2} \ln \frac{0.875}{0.125} = \frac{1}{2} \ln 7 = \frac{1}{2}(1.945910) = \mathbf{0.972955}.$$

Now $F(x) = 0.693147 h_1(x) + 0.972955 h_2(x)$ with $h_1 = (+, +, -, -, -)$ and $h_2 = (+, +, +, +, -)$:

x	h_1	h_2	$F(x)$	y
1	+1	+1	+1.666102	+1
2	+1	+1	+1.666102	+1
3	-1	+1	+0.279808	-1
4	-1	+1	+0.279808	+1
5	-1	-1	-1.666102	-1

Signs are $+, +, +, +, -$, so the ensemble gets $x = 1, 2, 4, 5$ right and $x = 3$ wrong: **four of five**. Both individual stumps also got four of five, so two rounds have not yet improved on a single stump here — which is honest and worth noticing. The reason is visible in the table: h_1 and h_2 disagree only on $x = 3$ and $x = 4$, and $\alpha_2 > \alpha_1$ so h_2 wins both. Points 3 and 4 are adjacent in x with opposite labels, and no combination of two thresholds can separate them from their neighbours; a third round is needed.

5. The decorrelation identity. (a) $\rho + (1 - \rho)/B = 0.40 + 0.60/200 = 0.40 + 0.0030 = \mathbf{0.4030}$.

(b) At $B = 2000$: $0.40 + 0.60/2000 = 0.40030$. The gain is 0.00270 — ten times the trees for a variance reduction of about a quarter of one percent.

(c) At $\rho = 0.20$, $B = 200$: $0.20 + 0.80/200 = \mathbf{0.2040}$. The gain is 0.1990, roughly **seventy-four times** the gain from the ten-fold increase in B .

Decorrelating is overwhelmingly the better investment, and that single comparison is the entire justification for `max_features`. Once B is in the hundreds, the $(1 - \rho)/B$ term is already negligible and all that is left is $\rho\sigma^2$.

6. Reading an OOB score. Two reasons the conclusion does not follow.

First, **one split is one observation**. The held-out score here is computed on 171 rows, so its standard deviation across splits is 0.0137 (Section 3.4); a gap of 0.0300 is about two standard deviations of a single noisy measurement, which happens regularly. Repeated over twenty splits the mean gap was +0.0008 with $p = 0.836$.

Second, the direction of any systematic OOB bias is *pessimistic*, not optimistic: each OOB prediction uses only the $\approx 37\%$ of members that did not see the row, so OOB measures a smaller ensemble than the one you deploy. There is no mechanism that would make OOB systematically flatter you.

To settle it: repeat the split with twenty or thirty different `random_state` values, collect both scores each time, and do a paired t -test on the differences, as in Section 3.4.

7. Reading an importance table. `transaction_id` is an identifier: it takes a distinct value on every row, carries no information about anything, and is the exact case Section 6 was written about. `timestamp` is close behind — thousands of distinct values, and very likely a leak besides. That these two top the list is not evidence that they matter; it is evidence that the metric is MDI. Note also that `is_weekend`, a two-level column, is at the bottom *by construction*: two levels means two candidate splits.

Ask for one computation: `permutation_importance` on a **held-out** split, with `n_repeats=30`, reporting the mean and the standard deviation. Expect `transaction_id` to come out at approximately 0.0000 with a tiny standard deviation, as `noise_uid(569)` did in Demonstration 1. Then have them drop the identifier and refit; if the accuracy is unchanged, the case is closed.

8. Which ensemble, and why. A random forest, and it is not close.

Pairwise annotator agreement of 85% means the label column carries double-digit noise. Section 8 measures exactly this: at 10% flipped labels the forest led AdaBoost by 0.0374 and at 20% by 0.0579, and with 25% flipped labels boosting peaked at $T = 19$ and then gave back 0.1345 while its training accuracy climbed to 0.9648 on labels that were a quarter wrong. Boosting’s reweighting rule cannot distinguish “hard row” from “wrong label”.

The other two constraints point the same way. One afternoon is not a tuning budget, and gradient boosting has at least three interacting hyperparameters (η , depth, T) while a forest has essentially none — the defaults were at or near the top of every column of the three-dataset table. And sixteen cores are worth sixteen times as much to a forest, whose trees are independent, as to boosting, which is inherently sequential. Use `oob_score=True` so the whole 50 000 rows train the model.

9. Gradient boosting by hand. $F_0 = \bar{y} = (2+4+8+10)/4 = 6$. Residuals: $-4, -2, +2, +4$. Initial SSE = $16 + 4 + 4 + 16 = 40$.

Try the three splits, computing the SSE of the residuals about their leaf means:

split	left leaf (mean)	right leaf (mean)	SSE of residuals
$x < 1.5$	$\{-4\} (-4)$	$\{-2, 2, 4\} (1.3333)$	$0 + 24.6667 = 24.6667$
$x < 2.5$	$\{-4, -2\} (-3)$	$\{2, 4\} (3)$	$2 + 2 = 4$
$x < 3.5$	$\{-4, -2, 2\} (-1.3333)$	$\{4\} (4)$	$24.6667 + 0 = 24.6667$

The stump splits at $x < 2.5$ with leaf values -3 and $+3$. So

$$F_1 = 6 + (-3, -3, +3, +3) = (3, 3, 9, 9),$$

and the new residuals are $2 - 3 = -1$, $4 - 3 = +1$, $8 - 9 = -1$, $10 - 9 = +1$, giving

$$\text{SSE} = 1 + 1 + 1 + 1 = 4.$$

One stump took the error from 40 to 4, a reduction of 90% — and note that the residual SSE after the split is exactly the “SSE of residuals” column, which is what the tree was minimising.

10. Why bagging fails on OLS. Bagging only moves the variance term of equation (4), and on a large dataset an OLS fit has almost no variance to move. The coefficient estimator $\hat{\beta} = (X^\top X)^{-1} X^\top y$ has covariance $\sigma^2 (X^\top X)^{-1}$, which shrinks like $1/n$; with n large, a bootstrap resample produces very nearly the same $\hat{\beta}$, so the members of the bag are almost identical — ρ near 1 in equation (5), and Section 2.4’s last row applies. Meanwhile the bias of a linear model on a non-linear truth is untouched, because bagging cannot touch bias.

This is Breiman’s own condition restated: bagging helps *unstable* predictors, and OLS with $n \gg p$ is about as stable as a method gets.

Bagging a linear model *would* help when the fit is unstable: p comparable to or larger than n , severe multicollinearity, or a fitting procedure with a discrete choice in it. Breiman’s 1996 paper makes exactly this point with *subset selection* in linear regression — the choice of which variables enter is a discontinuous function of the data, so it jumps around under resampling, and bagging it does help.

12 References and further reading

All links were checked on 23 August 2026. Free resources are marked [free].

Textbooks

- Han, Kamber and Pei, *Data Mining: Concepts and Techniques*, 3rd edition, Morgan Kaufmann, 2012. **The prescribed text for this lecture.** §8.6 techniques to improve classification accuracy — §8.6.1 introduces ensemble methods and the vocabulary the examination uses; §8.6.2 is bagging, with the algorithm in the same form as Section 3; §8.6.3 is boosting and AdaBoost, including the weight update of equation (7); §8.6.4 is random forests; §8.6.5 is class-imbalanced data. §8.5.4 on model selection and the paired t -test is the background for Section 3.4.
- James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python*. [free] §8.2 is this lecture almost exactly — §8.2.1 bagging, with the 2/3 figure and out-of-bag error estimation; §8.2.2 random forests and the decorrelation argument of Section 5; §8.2.3 boosting and its three tuning parameters; §8.2.5 variable importance. §5.2 is the bootstrap itself. <https://www.statlearning.com/>
- Müller and Guido, *Introduction to Machine Learning with Python*, O’Reilly, 2016. §2.3.6 ensembles of decision trees — random forests (building them, analysing them, the strengths and weaknesses section) and gradient boosted regression trees, with the `learning_rate` discussion of Section 7. Notebooks [free]: https://github.com/amueller/introduction_to_ml_with_python
- Hastie, Tibshirani and Friedman, *The Elements of Statistical Learning*, 2e, Springer. [free] §8.7 bagging; Chapter 10 on boosting, with §10.1 giving the AdaBoost.M1 algorithm exactly as Section 7 states it and §10.4 showing that it is forward stagewise fitting of an exponential loss; Chapter 15 on random forests, with §15.2 the definition, §15.4.1 the variance-and-decorrelation argument behind equation (5), and §15.3.2 variable importance. <https://hastie.su.domains/ElemStatLearn/>
- Zhi-Hua Zhou, *Ensemble Methods: Foundations and Algorithms*, Chapman and Hall/CRC, 2012. The monograph on the subject. Chapter 2 boosting, Chapter 3 bagging, Chapter 4 combination methods, Chapter 5 diversity — Chapter 5 is the formal treatment of the correlation argument this lecture makes empirically in Section 2.4.

Online courses

- NPTEL, **Introduction to Machine Learning** — Prof. Balaraman Ravindran, IIT Madras; the reference course for the semester. The ensembles week covers bagging, boosting, AdaBoost and random forests in the notation used here. [free] <https://nptel.ac.in/courses/106106139>

- **NPTEL, Data Mining** — Prof. Pabitra Mitra, IIT Kharagpur. Follows the Han–Kamber–Pei chapter structure, so its classification weeks map onto §8.6 directly. [free] <https://nptel.ac.in/courses/106105174>
- **NPTEL, Introduction to Machine Learning** — Prof. Sudeshna Sarkar, IIT Kharagpur. A second treatment of ensembles worked at the board, if the first does not land. [free] <https://nptel.ac.in/courses/106105152>
- **SWAYAM** — the national portal hosting NPTEL, with certification cycles. [free] <https://swayam.gov.in/>
- **Kaggle Learn, Intro to Machine Learning** — lesson 6 is random forests in the browser, five minutes after the decision-tree lessons. [free] <https://www.kaggle.com/learn/intro-to-machine-learning>
- **Google Machine Learning Crash Course** — the “Overfitting” and “Generalization” modules are the background for Section 8. [free] <https://developers.google.com/machine-learning/crash-course>

Videos

- StatQuest with Josh Starmer, *StatQuest: Random Forests Part 1 - Building, Using and Evaluating* — bagging, the random feature subset and the out-of-bag estimate at a whiteboard. Watch this one first. [free] https://youtu.be/J4WdyOWc_xQ
- StatQuest with Josh Starmer, *Bootstrapping Main Ideas!!!* — the bootstrap on its own, before any ensemble, which is the right order if Section 3.2 did not land. [free] <https://youtu.be/Xz0x-8-cgaQ>
- StatQuest with Josh Starmer, *AdaBoost, Clearly Explained* — the stumps, the amount of say (his name for α_t) and the reweighting, drawn out step by step. It is the video companion to Section 7.3. [free] <https://youtu.be/LsK-xG1cLYA>
- StatQuest with Josh Starmer, *Gradient Boost Part 1 (of 4): Regression Main Ideas* — fitting the residuals, with the learning rate. Parts 2 to 4 do the mathematics and the classification case. [free] <https://youtu.be/3CC4N4z3GJc>

Documentation and interactive explainers

- MLU-Explain, *The Random Forest Algorithm* — Amazon’s scrollable explainer, and it opens with Condorcet’s jury theorem exactly as Section 2 does, with sliders for the number of trees and each tree’s accuracy. The “Variance in Predictions” grid is Section 2.4 made visual. [free] <https://mlu-explain.github.io/random-forest/>
- MLU-Explain, *The Bias Variance Tradeoff* — the decomposition of equation (4) with interactive model realisations, which is what Figure 4 shows statically. [free] <https://mlu-explain.github.io/bias-variance/>
- scikit-learn user guide, 1.11. *Ensembles: Gradient boosting, random forests, bagging, voting, stacking* — the reference for every class used in these notes, including the SAMME weight that explains the factor of two in Section 7. [free] <https://scikit-learn.org/stable/modules/ensemble.html>
- scikit-learn user guide, 5.2. *Permutation feature importance* — §5.2.1 is the algorithm of Section 6, §5.2.2 states the high-cardinality bias of impurity importance in the documenta-

tion’s own words, and §5.2.3 is the correlated-features caveat. [free] https://scikit-learn.org/stable/modules/permutation_importance.html

- scikit-learn example, *Permutation Importance vs Random Forest Feature Importance (MDI)* — the same demonstration as Section 6, on the Titanic data with a planted random categorical column. [free] https://scikit-learn.org/stable/auto_examples/inspection/plot_permutation_importance.html

Articles

- L. Breiman, “Bagging predictors”, *Machine Learning* **24**(2), 123–140, 1996. The paper that named bagging. Its abstract contains the sentence Section 3 quotes — *the vital element is the instability of the prediction method* — and §4 is the subset-selection experiment behind the answer to Practice Question 10. <https://doi.org/10.1007/BF00058655>
- L. Breiman, “Random Forests”, *Machine Learning* **45**(1), 5–32, 2001. The abstract states the result of Section 5 directly: the generalisation error “depends on the strength of the individual trees in the forest and the correlation between them”, and a random feature selection gives error rates comparable to AdaBoost “but more robust with respect to noise” — which is what Section 8 measures. <https://doi.org/10.1023/A:1010933404324>
- Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting”, *Journal of Computer and System Sciences* **55**(1), 119–139, 1997. The AdaBoost paper. §3 is the algorithm of Section 7 and the derivation of α_t as the minimiser of the normaliser Z_t . <https://doi.org/10.1006/jcss.1997.1504>
- J. H. Friedman, “Greedy function approximation: a gradient boosting machine”, *Annals of Statistics* **29**(5), 1189–1232, 2001. The paper that made “fit the residuals” into “descend the gradient in function space”; §3 is the squared-error case worked in Section 7 and §5 introduces the shrinkage parameter η . <https://doi.org/10.1214/aos/1013203451>
- C. Strobl, A.-L. Boulesteix, A. Zeileis and T. Hothorn, “Bias in random forest variable importance measures: illustrations, sources and a solution”, *BMC Bioinformatics* **8**:25, 2007. [free] The formal version of Section 6: why impurity importance prefers high-cardinality predictors, and what to do instead. <https://doi.org/10.1186/1471-2105-8-25>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.; Zhang et al., *Dive into Deep Learning*, Ch. 5.